LogicDAG: Enhancing Logical Reasoning in LLMs via Solver-Verified Proof DAG Synthesis

Duc-Hieu Ho[1], Son N. Tran[2], Cuong Vo[3], Phuc-Tinh Pham Do[3], Tran Van Nhiem[4], Ngoc-Thanh Dinh[5], Thin Nguyen[2], Thanh-Toan Do[1], Trung Le[1]
[1]Monash University, [2]Deakin University, [3]VBI Academy
[4]AI Research Center, Hon Hai Research Institute, [5]VinUniversity

Abstract

Synthetic corpora for multi-step deduction rarely verify the inference steps they teach: most build their proofs from inference rules that are valid by design, label whole problems with a prover rather than check their steps, or let a language model rewrite the checked formulas as text. We introduce **LogicDAG**, a data synthesis pipeline in which solvers verify the logic of every item and a language model writes its text, and the corpus of logical reasoning items it produces. Each item is grounded in a proof directed acyclic graph (DAG) over propositional logic or first-order logic, in which a derived lemma can serve as a premise of several later inferences. In this DAG, a solver must verify each inference before a later inference may build on it. The language model gives each atom its meaning and rewrites the reasoning chain from a scaffold of one templated sentence per inference. In contrast, templates compose every premise and conclusion from the verified formulas, so their logical connectives are fixed by the proof. The *True*, *False*, and *Uncertain* labels likewise follow from the verified formulas by construction. The same model, prompted as a verifier, checks the text against the proof, and a quality gate then checks it against the formulas. Every item is released with its formal side: the formulas, the proof DAG, and the atom map it was built from. Fine-tuning instruction-tuned Llama models on the corpus improves formal deduction in propositional logic and first-order logic on external benchmarks, and the gains extend to mathematics benchmarks. We release the corpus at https://github.com/LogicDAG/LogicDAG.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; *Knowledge representation and reasoning*.

Keywords

Logical Reasoning, Dataset, Synthetic Data, Proof DAG, Large Language Models

1 Introduction

Large language models (LLMs) still fall short on multi-step deduction benchmarks [7, 17, 22], although logical reasoning underlies the mathematics, code generation, and scientific problem solving they are used for [9]. Fine-tuning on synthetic deductive corpora is one response. Since a model learns inference from the examples of such a corpus, the corpus has to meet two requirements: every inference step it contains is valid, and the text of every item states the logic that was verified.

Existing synthetic corpora meet the first requirement by design rather than by verification. They build a symbolic proof first and ground it in natural language afterwards: early corpora construct deductive chains by applying inference rules to a fixed set of facts and render them with templates [1, 2, 25], and later work synthesizes multi-step deductive corpora from formal logic at scale [15, 24]. These works show that training on such structured data improves deductive reasoning, from fine-tuned transformers to LLMs [16]. In these corpora, the inference rules are valid by design, or a prover labels each whole problem [24]. However, no automated theorem prover verifies the individual inference steps that an item teaches. The validity of each step therefore rests on the generator alone, and an error in the implementation of a rule would go unnoticed. The second requirement concerns the text: when an LLM writes the premises from the formulas, it can change their logic, and the item then teaches an unverified step. These observations motivate a corpus in which a solver verifies every inference step, and in which the logic of the text comes from the verified formulas rather than from the model.

Verifying every inference step requires a representation in which each step is a separate unit, and a proof DAG is such a representation: a directed acyclic graph whose nodes are formulas and whose edges record the premises of each inference. A derived formula that is a premise of more than one later inference is a lemma: it is derived once and applied at several later steps. Such DAGs already appear in prior systems. In AlphaGeometry [26], the symbolic deduction engine produces a directed acyclic graph of all reachable conclusions and extracts the minimal certifying subgraph with a traceback algorithm that relies on these shared derivations. Automated theorem provers and formal verification systems likewise record proofs as DAGs of single inferences. At this granularity, an automated theorem prover can verify every step on its own, which is the property a solver-verified corpus needs.

In this paper, we present **LogicDAG**, a synthesis pipeline built around solver-verified proof DAGs, and the corpus of multi-step deduction items in propositional logic and first-order logic that it produces (Figure 1). LogicDAG separates the logic of an item from its wording: solvers alone decide logical validity on the proof DAG, and an LLM writes the natural language. The first requirement, that every step be valid, is met on the formal side. A solver must verify every forward inference before any later inference may build on it: Z3 [4] and Vampire [11] both verify each propositional logic inference, while Vampire verifies each first-order logic inference. The validity of each step therefore rests on the solvers, so an error in the implementation of a rule is caught before the step enters the proof. *True*, *False*, and *Uncertain* items are then derived from the verified formulas by construction, and Z3 re-derives every label from the formulas released with the item.

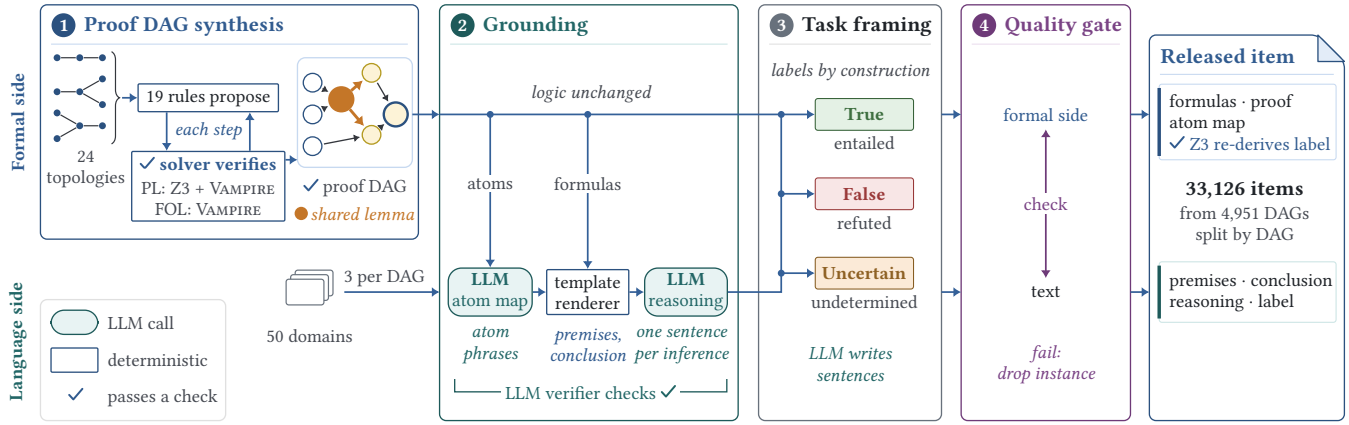


Figure 1: LogicDAG keeps the logic of an item apart from its wording. The upper row holds the formal side of an item, and the lower row holds its text. In Stage 1, one of the 24 topologies fixes the shape of a proof, the 19 inference rules propose its steps, and a solver verifies each step before a later step may use it. A derived formula that several later steps use is a shared lemma (orange). In Stage 2, the LLM gives each atom a phrase and writes one sentence per inference, templates compose every premise and conclusion from the verified formulas, and the same model, prompted as a verifier, then checks the text against the proof. In Stage 3, every label follows from the formulas by construction. Stage 4 checks the text against the formal side and drops each failing instance. Every released item carries its formal side, from which Z3 re-derives its label. Rounded teal boxes are LLM calls, square navy boxes are deterministic steps, and a tick marks a passed check.

Because a solver checks each inference on its own, the shape of a proof can be designed in advance without risk to its validity. A topology fixes the shape of each proof before any formula is chosen: linear topologies give depth, fan-out topologies apply one derived formula in several branches, fan-in topologies join two lines of argument, and compositions of these give mixed shapes. Since a conclusion reached along different paths is stored as a single node, a derived formula can serve as a lemma, and 1,311 of the 4,951 released DAGs (26.5%) contain one.

Three further mechanisms tie the text that the LLM writes to the verified proof. First, templates compose every premise and conclusion from the verified formulas, so their connectives are fixed by the proof. The same templates also write one sentence per inference of the proof, and the LLM only rewrites these sentences into a fluent reasoning chain, still one sentence per inference. Second, the same model, prompted as a verifier, checks each instance against its proof, where an instance is one proof DAG grounded in one domain. Third, if the text of an instance fails a check against its formulas, a quality gate removes the instance and all of its items. Each instance that passes these checks yields up to three items, and an item consists of premises, a conclusion, a reasoning chain, and a label. In addition, every released item carries its formal side: the formulas, the proof DAG, and the atom map it was built from.

We use the corpus to ask whether fine-tuning on LogicDAG raises accuracy on external logical reasoning benchmarks, in propositional logic and in first-order logic, with the base model as the reference throughout. We fine-tune three instruction-tuned models of the Llama 3 family (1B–8B) on the training split and compare each with its base model (§4.2). Among the external benchmarks, LogicBench MCQA, Multi-LogiEval [18], and FOLIO pose premises and ask what follows, as the corpus does. We therefore summarize them by the formal deduction average, the unweighted mean of the accuracies on the propositional logic and first-order logic items of LogicBench MCQA and Multi-LogiEval and on FOLIO. This average rises above the base model for each of the three models. In particular, we report the items of LogicBench MCQA and Multi-LogiEval in propositional logic and first-order logic separately, and accuracy on both benchmarks rises in both logics for every model. On Multi-LogiEval, accuracy also rises at almost every reasoning depth. The corpus thus transfers the deduction it trains in both logics. The gains also extend to LogiQA2.0 [14], LogicAsker [27], and AGIEval LSAT-LR [32], and to mathematics benchmarks.

Our contributions are threefold:

(1) **Solver-verified synthesis.** We propose LogicDAG, a pipeline in which a solver verifies every inference of a proof DAG in propositional logic or first-order logic, and templates compose every premise and conclusion from the verified formulas, so the premises and conclusions state the verified logic (§3).

(2) **A training corpus with its formal side.** We release the LogicDAG corpus as synthetic training data for improving the logical reasoning of language models. It holds *True*, *False*, and *Uncertain* items in propositional logic and first-order logic and is split by proof DAG. Each of these items carries its formal side: the formulas, the proof DAG, and the atom map it was built from (§4.1).

(3) **Empirical results.** Fine-tuning instruction-tuned Llama models on LogicDAG improves their logical reasoning: accuracy on formal deduction rises in propositional logic and first-order logic, and the gains extend to other logical reasoning benchmarks and to mathematics (§4).

2 Related Work

Deductive reasoning benchmarks. Benchmarks of deductive reasoning are built to measure whether a model reasons, and their findings motivate training on deduction. Clark et al. [2] test whether a transformer reasons over facts and rules stated in natural language, and ProofWriter [25] adds generated implications and their proofs for multi-step deduction. On such data, Bao et al. [1] study how multi-step deductive reasoners generalize to shuffled rules and greater reasoning depths, and LLMs prompted with chain-of-thought generalize to compositional proofs but not to longer ones [23]. Analyses of chain-of-thought outputs find that LLMs make valid individual deduction steps but struggle to choose which step to take [22], and that LLMs inducing logic theories struggle most with long chains of predicate relations [5]. Later benchmarks control the complexity and the validity of their tasks. AutoLogi [33], SATBench [28], and Multi-LogiEval [18] synthesize tasks of controllable complexity, Xu et al. [29] validate generated tasks with logic-programming test oracles, and FOLIO [7] pairs human-written problems in first-order logic with their formulas. Unlike these benchmarks, LogicDAG is built to train deduction rather than to measure it.

Synthetic training corpora. Corpora built for training check their steps at a coarser level than a solver-verified corpus requires. FLD [15] generates proofs from deduction rules that are valid by construction and renders them with templates, and its successor [16] shows that fine-tuning LLMs on such program-generated corpora raises their accuracy on logical reasoning, mathematics, and coding benchmarks. Sileo [24] samples premises and hypotheses from a grammar and labels each whole problem with the Vampire prover. ORACLE [30] has an LLM write each reasoning step and translate it into symbols, scores the step by whether a symbolic engine executes it and by LLM judgments, and pairs executed steps with failed ones as preference data. LogicPro [10] guides LLM-written reasoning with the intermediate values of an executed program. Outside logic, MetaMath [31] bootstraps mathematical questions by LLM rewriting and keeps solutions with correct answers. In these corpora a step counts as valid by construction, by the final label, or by a score, and no solver verifies each inference. LogicDAG differs in that a solver verifies every forward inference of a proof DAG. As in FLD, templates then compose every premise and conclusion from the verified formulas, so the premises and conclusions state the verified logic.

Prover-verified synthesis. Pipelines that call a symbolic prover during generation leave the text to an LLM. ProverGen [19] builds a reasoning tree top-down from a goal and uses a prover to compute the truth values of its premises at each layer. An LLM then translates each rule and fact into natural language, and each translation is checked for the names and predicates it must contain. Divide and Translate [21] translates problems into first-order logic compositionally and verifies the translation. LogicDAG instead builds each proof forward, has a solver verify every inference, and lets templates rather than the LLM write the premises and conclusions.

3 Method

In LogicDAG, solvers decide the validity of every inference on the proof DAG, an LLM gives each atom its meaning and writes the reasoning chain, and templates compose every premise and conclusion from the verified formulas. The pipeline comprises four stages. Stage 1 builds a proof DAG from designed proof topologies and verifies every forward inference with a solver before the inference enters the proof state (§3.1). Stage 2 grounds the verified proof DAG in natural language by giving its atoms meanings in a domain and writing its inferences as sentences, and the same model, prompted as a verifier, checks their faithfulness (§3.2). Stage 3 derives up to three labeled items from each instance: the verified conclusion for *True*, its logical negation for *False*, and a third item that asks about an underdetermined conclusion or a fresh clause (§3.3). Stage 4 removes every instance that fails a check against its formulas (§3.4). Figure 1 gives an overview of the pipeline.

3.1 Stage 1: Solver-Verified Symbolic Proof DAG Synthesis

Stage 1 synthesizes a proof DAG, a directed acyclic graph whose nodes are logical formulas and whose edges record the premises of each forward inference. Appendix A.4 gives an example proof DAG (Figure 4).

Logical language. The generator operates over a formula language spanning propositional logic and a fragment of first-order logic. The propositional logic fragment includes atomic propositions and the connectives $\neg$, $\wedge$, $\vee$, $\rightarrow$, and $\leftrightarrow$. The first-order logic fragment adds unary predicates, universal quantification, and existential quantification. Commutative connectives are canonicalized by sorting their arguments, so a conclusion reached along different paths is a single node of the DAG and can serve as a shared lemma.

Forward inference rules. The proof system has 19 forward inference rules: modus ponens, modus tollens, hypothetical syllogism, disjunctive syllogism, conjunction introduction and elimination, composition, double-negation elimination, constructive dilemma, destructive dilemma, bipolar dilemma, De Morgan, contraposition, material implication, importation, biconditional elimination in both directions, universal instantiation, and existential generalization. Appendix A.1 details each rule.

Each rule is implemented as a generator that draws candidate triples of conclusion, premises, and rule name from the current proof state. The solver then decides whether a candidate enters the proof state by checking that the premises entail the candidate conclusion.

Proof topologies. Proof structures come from a topology library and are fixed before any formula is instantiated. The library has seven base topologies. Of these, six pair each of the two logics with one of three structural primitives: linear, fan-out, and fan-in. The seventh chains universal implications in first-order logic without grounding them at a constant. Among the primitives, linear topologies produce sequential multi-step reasoning chains. Fan-in topologies construct two independent proof arms that converge at a merge rule, such as a dilemma, conjunction introduction, or modus

ponens. Fan-out topologies, in contrast, derive a shared lemma that two or more independent downstream branches reuse.

A composition feeds a formula derived by one topology into a second topology. Composing pairs of the six topologies built from a structural primitive yields 17 compositions, so the library holds 24 topologies. The compositions fall into three groups by their second topology. First, six compositions end in a fan-out. In these, the first topology derives a lemma that the fan-out distributes across branches, so nearly every DAG whose topology ends in a fan-out reuses a derived lemma (Appendix A.2). Second, six compositions end in a linear topology and produce deeper proofs without increasing the branching degree. Third, five compositions end in a fan-in, so a formula derived by the first topology becomes one arm of a convergence.

Rule compatibility graph. Within the propositional logic topologies, the rule sequence is drawn from a typed compatibility graph, where each node is an inference rule and each directed edge connects a rule to a rule that can take its output as input. For example, implication outputs feed modus tollens, hypothetical syllogism, or contraposition, while disjunction and conjunction outputs feed disjunctive syllogism and conjunction elimination, respectively. A random walk over this graph samples a rule sequence, and the topology constructor then builds a seed set such that applying this sequence in forward order derives the target. The first-order logic topologies instead instantiate fixed proof patterns in which one rule is drawn at random. Appendix A.2 lists all edges and describes the first-order logic proof patterns.

Forward saturation and proof extraction. Starting from the seed axioms, forward saturation applies all inference rules to the accumulated proof state in repeated passes. In each pass, candidate conclusions are collected from all current facts, any candidate already in the proof state is skipped at collection time, and each remaining candidate must pass a per-inference solver check before insertion. Saturation halts when the target formula appears in the proof state, when the proof state reaches a size bound, when a pass adds no new facts, or after a pass limit set by the depth of the target. Prior to saturation, each seed set is also verified to be satisfiable, since inconsistent premises would trivially entail any conclusion. Once saturation completes, proof extraction performs a depth-first traversal from the target node through its premise links and collects only the nodes that contribute to deriving the target. A shared lemma therefore remains a single node of the extracted DAG. An attempt that misses the target, or that yields a trivial or duplicate proof, is discarded.

Entailment verification. Every inference in Stage 1 is verified before its conclusion enters the proof state. Inferences in propositional logic are verified by both Z3 [4] and Vampire [11] in a cross-prover configuration, in which a disagreement rejects the candidate. In contrast, any inference involving a predicate or quantifier is routed to Vampire, since the Z3 check of the pipeline encodes formulas of propositional logic only. If Vampire times out or returns an inconclusive result, the candidate is rejected. The solvers check each proof twice. First, they check every forward inference at its insertion, as described above. Second, after proof extraction, they check that the recovered seed set entails the target and not its negation. Together, the two checks make each step and the assembled proof sound. In the run that produced the corpus, the two solvers agreed on all 2,166,196 checks in propositional logic. In addition, no inference proposed by a rule was rejected, so the solvers found no faulty rule. After generation, a separate post-check with its own formula parser re-verified all 37,345 inferences together with the entailment and the label of every DAG, and all 5,000 DAGs passed. Table 1 reports the solver checks for the 5,000 generated DAGs, and Appendix A.3 details them.

Table 1: Solver entailment checks in the run that generated the 5,000 proof DAGs. Z3 and Vampire both check each inference in propositional logic (PL), and Vampire alone checks each inference in first-order logic (FOL).

	Z3	Vampire
PL checks	2,166,196	2,166,196
FOL checks	—	883,553
Total	2,166,196	3,049,749

3.2 Stage 2: Natural Language Grounding

Stage 2 grounds each verified proof DAG in natural language in three phases, all run by one LLM, Qwen3.8-27B [20][1]. Each proof DAG is grounded in three distinct domains drawn at random, which gives three instances per proof DAG. The 50 domains span everyday life, the natural sciences, the social sciences and humanities, technology and engineering, economics and business, and geography and public affairs, so that each instance reads as a plausible argument. Appendix B.1 lists them. The premises of an instance are statements about a scenario in its domain, to be taken as given rather than checked against world knowledge. In Phases 2a and 2b, the LLM produces the atom map and the reasoning chain. In Phase 2c, the same model, prompted as a verifier, checks the generated text against the proof. Appendix B.4 gives the system prompts of the three phases, and Appendix B.2 grounds the proof DAG of Figure 4, with its shared lemma marked (Figure 5).

Phase 2a: Atom mapping. The LLM maps each atom to a phrase in the sampled domain and to its natural negation. A topology hint that describes the pattern of the proof guides this mapping, so that every logical implication reads as a plausible rule of one scenario. For DAGs in first-order logic, it also maps each predicate to a property and maps each constant to an entity. The output is checked so that every atom has a phrase and no phrase contains a connective, a negation, or a hedge. Between Phases 2a and 2b, a recursive template renderer writes every formula in natural language by traversing its abstract syntax tree, so connectives and quantifiers are worded uniformly across all premises, the target conclusion, and the proof scaffold. The connectives of every premise and conclusion therefore come from the verified formulas, and the LLM chooses only what the atoms mean. The renderer states some conditional premises as the knowledge of a person in the scenario, which changes the wording and not the formula.

[1] https://huggingface.co/Qwen/Qwen3.8-27B

Phase 2b: Reasoning chain. The renderer builds a scaffold from the proof DAG, one templated sentence per inference that names the rule with its rendered premises and conclusion. The LLM then rewrites the scaffold into a fluent reasoning chain, still one sentence per inference, and the output is checked to have as many sentences as the proof has inferences.

Phase 2c: Faithfulness check. In a separate call with a verification prompt, the same model receives the atom map and the rule-labeled scaffold together with the generated text. It checks three properties of each instance jointly: the text uses the atom map consistently, each sentence of the reasoning chain is faithful to its inference, and the conclusion states the target. The verifier thus judges the finished instance as a whole, after every phrase and sentence has been written, and only an instance that receives a True verdict is admitted to the corpus. It rejects 1,131 of the 15,000 instances: 1,025 with a False verdict and 106 whose verdict could not be parsed (Table 2). The quality gate of Stage 4 then checks the admitted text again against fixed references. Appendix B.3 breaks down the rejections.

3.3 Stage 3: Task Framing

Stage 3 converts each admitted instance into up to three labeled items following the ternary label space of prior logical reasoning benchmarks [7, 25]: *True* when the premises entail the conclusion, *False* when the premises refute it, and *Uncertain* when the conclusion is logically underdetermined by the premises. All labels are derived mechanically from the verified formulas, and Z3 re-derives each label from the premise and conclusion formulas released with the item.

First, a *True* item pairs the original premises with the target conclusion and the reasoning chain. Second, a *False* item pairs the same premises with the logical negation of the target conclusion, which the same template renderer writes in natural language. Its reasoning chain adds a single LLM-generated refutation sentence to the *True* reasoning chain. Third, a base *Uncertain* item is constructed when the target admits an underdetermined variant: one disjunct of a disjunctive target, the converse of a conditional target, or the universal counterpart of an existential target. Its reasoning chain adds one or two sentences that identify the gap between what the premises establish and the proposed conclusion.

Base *Uncertain* items exist only for these three target shapes. A fresh-clause augmentation therefore adds at most one item to an instance that has no base *Uncertain* item. The LLM writes a new statement about the scenario, which enters the formulas as a new atom, so the label of the new item follows from the formulas alone. The augmentation has two kinds. In the *unmentioned* kind, the conclusion is the new statement or its negation, the premises are unchanged, and the label is *Uncertain*. In the *conditional* kind, one added premise links the new statement to a statement that the proof settles. Modus ponens and modus tollens then yield *True* or *False*, while affirming the consequent and denying the antecedent yield *Uncertain*. The LLM writes only sentences in Stage 3, so none of them can change a label. In addition, a second call checks each fresh clause, and Stage 4 removes an instance whose *False* or *Uncertain* reasoning chain claims that the conclusion follows. Figure 2 shows the items of one instance, Appendix C.1 follows their construction, and Appendix C.2 gives the prompts of Stage 3.

Table 2: Construction of the LogicDAG corpus, from proof DAG attempts to released splits. PL and FOL denote propositional logic and first-order logic, and parentheses give the counts by logic or the items of the removed instances.

Stage	Quantity	Count
1 Synthesis	Attempts	5,050
	Proof DAGs (PL / FOL)	5,000 (3,800 / 1,200)
2 Grounding	Instances generated	15,000
	Instances admitted	13,867
3 Framing	Framed items	33,618
4 Quality gate	Removed instances (items)	199 (492)
	Released items	33,126
	Released instances / DAGs	13,668 / 4,951
Split by DAG	Train / validation / test items	31,495 / 785 / 846
	Train / validation / test DAGs	4,705 / 122 / 124
Release	True / False / Uncertain items	14,125 / 14,100 / 4,901
	PL / FOL items	24,823 / 8,303

3.4 Stage 4: Quality Gate

Every item carries its *formal side*: the premise formulas, the conclusion formula, the proof DAG, and the atom map. The solvers' guarantees end at the formulas, so Stage 4 checks the text that Stages 2 and 3 write wherever a fixed reference exists: it compares the reasoning chain with the proof, it compares the wording with the atom map, and it evaluates the arithmetic statements of the text. An instance that fails a check is removed with all of its items, and no text is edited. Appendix D describes the checks, and §4.1 follows the run that produced the LogicDAG corpus through the four stages.

4 Experiments and Results

LogicDAG is built for deduction in propositional logic (PL) and first-order logic (FOL), so our experiments ask: does fine-tuning on its training split raise the accuracy of a base model on logical reasoning benchmarks? In §4.1, we describe the corpus that one run of the pipeline produced. In §4.2, we evaluate on external logical reasoning benchmarks, where fine-tuning raises accuracy on formal deduction in both propositional logic and first-order logic and the gains extend to mathematics.

4.1 LogicDAG Corpus

Table 2 follows the run that produced the LogicDAG corpus through the four stages of §3 and its split by proof DAG. One LLM, Qwen3.8-27B[1], wrote and checked the natural language of every instance. Two stages remove instances, and the larger loss comes first. In Stage 2, the verifier of Phase 2c rejects 1,131 of the 15,000 instances (7.5%), and two more instances fail generation. The Stage 4 quality gate then removes 199 instances (492 items).

The LogicDAG corpus balances *True* and *False* items, and *Uncertain* items account for the remaining 14.8% (Table 2). The fresh-clause augmentation of §3.3 contributes 2,204 of these *Uncertain*

Premises 1–4 (shared by all three items, Information Systems domain)

1.	The server receives the backup command	P_{954}
2.	If the server receives the backup command, then the backup job completes successfully	$P_{954} \rightarrow A_{791}$
3.	If the backup job completes successfully, then the database snapshot is created	$A_{791} \rightarrow A_{794}$
4.	If the backup job completes successfully, then the integrity check passes	$A_{791} \rightarrow A_{795}$

Shared reasoning chain: steps 1–4 of every response

1.	Since the server receives the backup command, and the stated rule indicates that receiving this command leads to a successful backup job, it follows that the backup job completes successfully.	MP: A_{791}
2.	Given that the backup job completes successfully, and the rule linking this completion to the creation of a database snapshot, we conclude that the database snapshot is created.	MP: A_{794}
3.	Since the backup job completes successfully, and the rule linking this completion to a passing integrity check, it follows that the integrity check passes.	MP: A_{795}
4.	Combining the established fact that the database snapshot is created with the fact that the integrity check passes, we conclude that the database snapshot is created and the integrity check passes.	CI: $A_{794} \wedge A_{795}$

↓ *same premises 1–4 and same steps 1–4: the items differ in their conclusion, and the False and Uncertain items add a fifth step*

***True* item**

Conclusion ($A_{794} \wedge A_{795}$, the target):

The database snapshot is created and the integrity check passes

Response:

Reasoning:
1.–4. [shared reasoning chain above]

Final verdict: True
Final answer: `\boxed{True}`

***False* item**

Conclusion ($\neg A_{794} \vee \neg A_{795}$, negation of the target):

Either the database snapshot is not created or the integrity check does not pass, or both

Response:

Reasoning:
1.–4. [shared reasoning chain above]

5. The proposed conclusion is false because the premises entail that the database snapshot is created and the integrity check passes, directly contradicting the disjunction.

Final verdict: False
Final answer: `\boxed{False}`

***Uncertain* item (fresh clause)**

Conclusion ($\neg C$):

The backup log is not archived to the remote storage

Response:

Reasoning:
1.–4. [shared reasoning chain above]

5. Nothing in the derivation above bears on whether the backup log is not archived to the remote storage; the premises decide the conclusion they reach and leave this further statement undetermined.

Final verdict: Uncertain
Final answer: `\boxed{Uncertain}`

Figure 2: The three items of one instance, a fan-out proof DAG in propositional logic grounded in the Information Systems domain. The top panels give the premises and reasoning chain that the three items share, and the bottom panels give what each item adds. The formal side of each line is printed beside it, and C denotes the fresh clause, a new atom. The derived formula A_{791} is the shared lemma of the proof, since steps 2 and 3 both use it.

items. Across the two logics, items in propositional logic account for 74.9% of the corpus, and items in first-order logic account for the remainder. In 1,311 of the 4,951 released DAGs (26.5%), a derived lemma is a premise of at least two inferences. Most of the released proofs are three to five inferences deep, and only proofs in propositional logic go beyond depth eight (Table 3). In addition, all 19 forward inference rules occur in the released proofs. Finally, each split covers all 24 topologies and all 50 domains, and no proof DAG occurs in more than one split, so a held-out item tests a proof that the model has not seen in training. Appendix E.1 gives the split and width distributions and the inference rule usage.

We also use the held-out test split to place the corpus among the external benchmarks by difficulty. We pose its 846 items, which come from 124 proof DAGs, to the three base models in the prompt format of the training items. Decoding is greedy, with a budget of 2,048 tokens. Figure 3 compares their accuracy on the split with that on the benchmarks of Table 4. Llama-3.2-1B-Instruct, Llama-3.2-3B-Instruct, and Llama-3.1-8B-Instruct reach accuracies of 36.29, 58.51, and 62.77, respectively, so accuracy rises with model size. Over the three models, the split is harder than LogicAsker and LogicBench MCQA, about as hard as Multi-LogiEval, and easier than FOLIO, AGIEval LSAT-LR, and LogiQA2.0. The split thus separates models

Table 3: Proof depth of the released proof DAGs and items, by logical fragment.

	DAGs				Items	
Depth	**PL**	**FOL**	**All**	**%**	**All**	**%**
2	568	168	736	14.9	5,498	16.6
3	1,017	225	1,242	25.1	8,278	25.0
4	1,032	372	1,404	28.4	9,253	27.9
5	612	314	926	18.7	5,917	17.9
6	294	114	408	8.2	2,637	8.0
7	135	5	140	2.8	932	2.8
8	60	1	61	1.2	395	1.2
9	25	0	25	0.5	160	0.5
10	5	0	5	0.1	32	0.1
11	2	0	2	<0.1	14	<0.1
12	2	0	2	<0.1	10	<0.1
Total	3,752	1,199	4,951	100.0	33,126	100.0

Table 4: Accuracy (%) of three Llama models before (Base) and after fine-tuning on LogicDAG, on six logical reasoning benchmarks. Higher is better. ↑ and ↓ give the change from Base in points. PL and FOL denote the items in propositional logic and first-order logic, and Avg. is the mean of the two.

	LogicBench MCQA			Multi-LogiEval			FOLIO	LogicAsker	LogiQA2.0	AGIEval
Model	PL	FOL	Avg.	PL	FOL	Avg.				LSAT-LR
Llama-3.1-8B-Instruct										
Base	78.75	82.22	80.49	64.57	60.56	62.57	58.62	83.92	48.79	51.18
LogicDAG	86.88 ↑ 8.13	83.33 ↑ 1.11	85.10 ↑ 4.61	69.90 ↑ 5.33	65.42 ↑ 4.86	67.66 ↑ 5.09	64.04 ↑ 5.42	84.12 ↑ 0.20	51.34 ↑ 2.55	53.33 ↑ 2.15
Llama-3.2-3B-Instruct										
Base	75.00	75.56	75.28	52.38	47.48	49.93	46.80	76.90	39.89	36.67
LogicDAG	77.50 ↑ 2.50	80.56 ↑ 5.00	79.03 ↑ 3.75	56.38 ↑ 4.00	51.96 ↑ 4.48	54.17 ↑ 4.24	44.33 ↓ 2.47	77.63 ↑ 0.73	43.64 ↑ 3.75	39.41 ↑ 2.74
Llama-3.2-1B-Instruct										
Base	43.12	43.89	43.51	34.48	40.19	37.33	35.96	61.27	20.55	22.35
LogicDAG	58.75 ↑ 15.63	56.11 ↑ 12.22	57.43 ↑ 13.92	52.57 ↑ 18.09	53.83 ↑ 13.64	53.20 ↑ 15.87	30.05 ↓ 5.91	69.62 ↑ 8.35	31.49 ↑ 10.94	23.73 ↑ 1.38

Table 5: Multi-LogiEval accuracy (%) by reasoning depth d_1 to d_5, for the models of Table 4. Higher is better. Boldface marks a fine-tuned score above Base.

	PL					FOL				
Model	d_1	d_2	d_3	d_4	d_5	d_1	d_2	d_3	d_4	d_5
Llama-3.1-8B-Instruct										
Base	73.33	60.95	62.22	65.83	53.33	73.08	60.95	57.04	64.17	24.44
LogicDAG	**76.67**	**73.33**	**67.41**	**69.17**	53.33	67.69	**67.62**	**67.41**	**66.67**	**44.44**
Llama-3.2-3B-Instruct										
Base	50.00	42.86	57.04	58.33	51.11	52.31	55.24	44.44	48.33	22.22
LogicDAG	**69.17**	**43.81**	**59.26**	51.67	**55.56**	**61.54**	**56.19**	**50.37**	48.33	**28.89**
Llama-3.2-1B-Instruct										
Base	47.50	37.14	31.11	26.67	24.44	62.31	35.24	33.33	31.67	31.11
LogicDAG	**73.33**	**45.71**	**48.15**	**42.50**	**53.33**	**70.77**	**57.14**	**44.44**	**47.50**	**42.22**

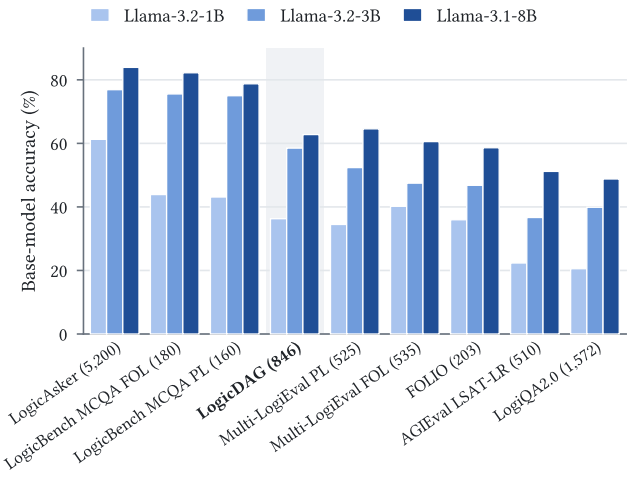


Figure 3: Accuracy of the three base models on the LogicDAG test split and the benchmarks of Table 4. The sets are sorted by the mean accuracy of the three models, easiest first, and parentheses give their number of items.

of different size without saturating them. Appendix E.4 breaks its items down by label, logical fragment, and depth.

4.2 Fine-tuning Raises Formal Deduction in Propositional Logic and First-Order Logic

Models and training. To test whether the corpus transfers, we fine-tune three open-weight instruction-tuned models of the Llama 3 family [6], at scales from 1B to 8B: Llama-3.1-8B-Instruct, Llama-3.2-3B-Instruct, and Llama-3.2-1B-Instruct. Each model receives full-parameter fine-tuning on the 31,495 items of the LogicDAG training split. All three models are trained with the same configuration for three epochs. Detailed hyperparameters are provided in Appendix E.2.

Benchmarks. We evaluate on external logical reasoning benchmarks, none of which LogicDAG is built from. Most of them pose premises and ask what follows from them, the task that LogicDAG trains. LogicBench [17] poses multiple-choice (MCQA) items, and we report its items in propositional logic (PL) and first-order logic (FOL). Multi-LogiEval [18] (depths 1–5) asks yes/no questions, which we also report separately for PL and FOL. FOLIO [7] poses problems in first-order logic and asks whether a conclusion is *True*, *False*, or *Uncertain*. LogicAsker [27] also asks yes/no questions, which cover rules of propositional logic and first-order logic. The remaining two benchmarks hold multiple-choice questions. LogiQA2.0 [14] draws them from civil-service exams, and AGIEval LSAT-LR [32] draws them from the logical reasoning section of the LSAT. Table 5 reports Multi-LogiEval by reasoning depth, and Table 6 reports three mathematics benchmarks. Base models and fine-tuned models are evaluated identically, with greedy decoding and the prompts and scoring rules of Appendix E.6.

Table 6: Accuracy (%) on three mathematics benchmarks, for the models of Table 4. Higher is better. ↑ and ↓ give the change from Base in points.

Model	GSM8K	MATH-500	AGIEval-MATH
Llama-3.1-8B-Instruct			
Base	83.09	46.60	45.60
LogicDAG	83.93 ↑ 0.84	49.60 ↑ 3.00	48.00 ↑ 2.40
Llama-3.2-3B-Instruct			
Base	72.86	46.20	38.50
LogicDAG	76.35 ↑ 3.49	43.00 ↓ 3.20	42.00 ↑ 3.50
Llama-3.2-1B-Instruct			
Base	35.41	22.20	18.10
LogicDAG	35.25 ↓ 0.16	25.20 ↑ 3.00	18.50 ↑ 0.40

Formal deduction improves in propositional logic and first-order logic. Table 4 reports each base model before and after fine-tuning on every benchmark. On LogicBench MCQA and Multi-LogiEval, whose items are split by logic, the fine-tuned model is above its base model in all six propositional logic cells and in all six first-order logic cells. The mean accuracy over the two logics rises by 3.75 to 13.92 points on LogicBench MCQA and by 4.24 to 15.87 points on Multi-LogiEval. The smallest model, Llama-3.2-1B-Instruct, gains most, by up to 18.09 points. To summarize these benchmarks together with FOLIO, we define the formal deduction average as the unweighted mean of the accuracies on LogicBench MCQA PL, LogicBench MCQA FOL, Multi-LogiEval PL, Multi-LogiEval FOL, and FOLIO. The formal deduction average rises above the base model for all three models, by 2.70 to 10.73 points. Within this average, accuracy on FOLIO rises by 5.42 points for Llama-3.1-8B-Instruct and falls by 2.47 and 5.91 points for Llama-3.2-3B-Instruct and Llama-3.2-1B-Instruct, respectively. These two drops come with responses that run to the generation limit of 1,024 tokens without an answer. Such responses account for 20 of the 47 FOLIO items that Llama-3.2-3B-Instruct loses against its base model and for 21 of the 45 items that Llama-3.2-1B-Instruct loses. The gains also hold across reasoning depth: the fine-tuned model is above its base model at almost every depth of both Multi-LogiEval subsets, and the smallest model is above its base model at every depth (Table 5). On LogicBench MCQA and Multi-LogiEval, which judge a conclusion against given premises under the rules of propositional logic or first-order logic, the corpus thus transfers the deduction it trains in both logics.

Gains extend to the other benchmarks. Accuracy on LogiQA2.0 and LogicAsker rises for all three models, by up to 10.94 and 8.35 points, respectively, both for Llama-3.2-1B-Instruct. AGIEval LSAT-LR mostly asks about the structure of informal arguments, such as their assumptions and flaws, rather than about what follows from given premises. Accuracy on it also rises for all three models, by 1.38 to 2.74 points. Excluding the averages, the fine-tuned model is above its base model in 22 of the 24 cells of Table 4.

Gains extend to mathematics. We also evaluate the fine-tuned models on three mathematics benchmarks: GSM8K [3] (1,319 test items), MATH-500 [13] (the 500-problem subset of MATH [8]), and AGIEval-MATH (the 1,000 MATH problems of AGIEval [32]). Fine-tuning on the corpus raises accuracy here as well: the fine-tuned model is above its base model in 7 of the 9 cells (Table 6), and the mean accuracy over the three benchmarks rises for every model, by 1.08 to 2.08 points.

Fine-tuning on LogicDAG thus raises accuracy on formal deduction in propositional logic and first-order logic, the deduction that the corpus is built for. The gains also extend to mathematics.

5 Conclusion

In this paper, we introduced LogicDAG, a pipeline that synthesizes solver-verified proof DAGs and grounds them in natural language, and the corpus of logical reasoning items it produces. The pipeline protects the logic of each item at four levels. First, a solver verifies each forward inference of the proof DAG. Second, templates compose every premise and conclusion from the verified formulas, and the renderer builds the scaffold of the reasoning chain from the verified steps. Third, the same model, prompted as a verifier, checks each instance against its proof. Fourth, a quality gate removes every instance whose text fails a check against its formulas. In addition, every label follows from the formulas by construction, Z3 re-derives it from the formulas released with the item, and every item is released with its formal side.

The LogicDAG corpus consists of 33,126 items in propositional logic and first-order logic, split by proof DAG, so a held-out item tests a proof that no training item shares. In addition, a quarter of its proof DAGs contain a lemma.

We fine-tuned three open-weight instruction-tuned models of the Llama 3 family (1B–8B) on the training split to test whether the deduction of the corpus transfers to external logical reasoning benchmarks. Accuracy rises for every model in both logics on LogicBench MCQA and Multi-LogiEval, and the formal deduction average rises for every model. The gains extend to LogiQA2.0, LogicAsker, AGIEval LSAT-LR, and mathematics benchmarks.

6 Limitations

LogicDAG verifies its inferences and labels with solvers. Its natural language is checked by an LLM verifier that reads the atom map together with the text, and then by the quality gate. Even so, a small number of wording errors may pass both checks. In addition, the formula language uses unary predicates, and templates fix the phrasing of every premise and conclusion. We leave relations of two or more arguments and freer phrasing to future versions of the pipeline. Due to limited computational resources, we only fine-tune small and mid-sized models.

References

[1] Qiming Bao, Alex Yuxuan Peng, Tim Hartill, Neset Tan, Zhenyun Deng, Michael Witbrock, and Jiamou Liu. 2025. Multi-Step Deductive Reasoning Over Natural Language: An Empirical Study on Out-of-Distribution Generalisation. arXiv:2207.14000 [cs.CL] https://arxiv.org/abs/2207.14000

[2] Peter Clark, Oyvind Tafjord, and Kyle Richardson. 2020. Transformers as Soft Reasoners over Language. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*, Christian Bessiere (Ed.). International Joint Conferences on Artificial Intelligence Organization, Yokohama, Japan, 3882–3890. doi:10.24963/ijcai.2020/537

[3] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 [cs.LG] https://arxiv.org/abs/2110.14168

[4] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (Lecture Notes*

in Computer Science), C. R. Ramakrishnan and Jakob Rehof (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 337–340. doi:10.1007/978-3-540-78800-3_24

[5] João Pedro Gandarela, Danilo S. Carvalho, and André Freitas. 2025. Inductive Learning of Logical Theories with LLMs: An Expressivity-Graded Analysis. arXiv:2408.16779 [cs.CL] https://arxiv.org/abs/2408.16779

[6] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] https://arxiv.org/abs/2407.21783

[7] Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, Lucy Sun, Alexander Wardle-Solano, Hannah Szabó, Ekaterina Zubova, Matthew Burtell, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, Rui Zhang, Alexander R. Fabbri, Wojciech Kryściński, Semih Yavuz, Ye Liu, Xi Victoria Lin, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Rex Ying, Arman Cohan, and Dragomir Radev. 2024. FOLIO: Natural Language Reasoning with First-Order Logic. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 22017–22031. doi:10.18653/v1/2024.emnlp-main.1229

[8] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, J. Vanschoren and S. Yeung (Eds.), Vol. 1. https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/file/be83ab3ecd0db773eb2dc1b0a17836a1-Paper-round2.pdf

[9] Jie Huang and Kevin Chen-Chuan Chang. 2023. Towards Reasoning in Large Language Models: A Survey. In *Findings of the Association for Computational Linguistics: ACL 2023*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 1049–1065. doi:10.18653/v1/2023.findings-acl.67

[10] Jin Jiang, Yuchen Yan, Yang Liu, Jianing Wang, Shuai Peng, Xunliang Cai, Yixin Cao, Mengdi Zhang, and Liangcai Gao. 2025. LogicPro: Improving Complex Logical Reasoning via Program-Guided Learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 26200–26218. doi:10.18653/v1/2025.acl-long.1270

[11] Laura Kovács and Andrei Voronkov. 2013. First-Order Theorem Proving and Vampire. In *Computer Aided Verification*, Natasha Sharygina and Helmut Veith (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 1–35. doi:10.1007/978-3-642-39799-8_1

[12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[13] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's Verify Step by Step. arXiv:2305.20050 [cs.LG] https://arxiv.org/abs/2305.20050

[14] Hanmeng Liu, Jian Liu, Leyang Cui, Zhiyang Teng, Nan Duan, Ming Zhou, and Yue Zhang. 2023. LogiQA 2.0—An Improved Dataset for Logical Reasoning in Natural Language Understanding. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 31 (2023), 2947–2962. doi:10.1109/TASLP.2023.3293046

[15] Terufumi Morishita, Gaku Morio, Atsuki Yamaguchi, and Yasuhiro Sogawa. 2023. Learning Deductive Reasoning from Synthetic Corpus based on Formal Logic. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*, Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.). PMLR, Honolulu, Hawaii, USA, 25254–25274. https://proceedings.mlr.press/v202/morishita23a.html

[16] Terufumi Morishita, Gaku Morio, Atsuki Yamaguchi, and Yasuhiro Sogawa. 2024. Enhancing Reasoning Capabilities of LLMs via Principled Synthetic Logic Corpus. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., Vancouver, BC, Canada, 73572–73604. doi:10.52202/079017-2340

[17] Mihir Parmar, Nisarg Patel, Neeraj Varshney, Mutsumi Nakamura, Man Luo, Santosh Mashetty, Arindam Mitra, and Chitta Baral. 2024. LogicBench: Towards Systematic Evaluation of Logical Reasoning Ability of Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 13679–13707. doi:10.18653/v1/2024.acl-long.739

[18] Nisarg Patel, Mohith Kulkarni, Mihir Parmar, Aashna Budhiraja, Mutsumi Nakamura, Neeraj Varshney, and Chitta Baral. 2024. Multi-LogiEval: Towards Evaluating Multi-Step Logical Reasoning Ability of Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 20856–20879. doi:10.18653/v1/2024.emnlp-main.1160

[19] Chengwen Qi, Ren Ma, Bowen Li, He Du, Binyuan Hui, Jinwang Wu, Yuanjun Laili, and Conghui He. 2025. Large Language Models Meet Symbolic Provers for Logical Reasoning Evaluation. In *The Thirteenth International Conference on Learning Representations*. Singapore, 54186–54205. https://openreview.net/forum?id=C25SgeXWjE

[20] Qwen Team. 2026. Qwen3.8-Max: A New Bar for Coding and Cowork. https://qwen.ai/blog?id=qwen3.8

[21] Hyun Ryu, Gyeongman Kim, Hyemin S. Lee, and Eunho Yang. 2025. Divide and Translate: Compositional First-Order Logic Translation and Verification for Complex Logical Reasoning. In *The Thirteenth International Conference on Learning Representations*. Singapore, 24935–24964. https://openreview.net/forum?id=09FiNmvNMw

[22] Abulhair Saparov and He He. 2023. Language Models Are Greedy Reasoners: A Systematic Formal Analysis of Chain-of-Thought. In *The Eleventh International Conference on Learning Representations*. Kigali, Rwanda. https://openreview.net/forum?id=qFVVBzXxR2V

[23] Abulhair Saparov, Richard Yuanzhe Pang, Vishakh Padmakumar, Nitish Joshi, Mehran Kazemi, Najoung Kim, and He He. 2023. Testing the General Deductive Reasoning Capacity of Large Language Models Using OOD Examples. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., New Orleans, Louisiana, USA, 3083–3105. doi:10.52202/075280-0136

[24] Damien Sileo. 2024. Scaling Synthetic Logical Reasoning Datasets with Context-Sensitive Declarative Grammars. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 5275–5283. doi:10.18653/v1/2024.emnlp-main.301

[25] Oyvind Tafjord, Bhavana Dalvi Mishra, and Peter Clark. 2021. ProofWriter: Generating Implications, Proofs, and Abductive Statements over Natural Language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.). Association for Computational Linguistics, Online, 3621–3634. doi:10.18653/v1/2021.findings-acl.317

[26] Trieu H. Trinh, Yuhuai Wu, Quoc V. Le, He He, and Thang Luong. 2024. Solving Olympiad Geometry without Human Demonstrations. *Nature* 625, 7995 (2024), 476–482. doi:10.1038/s41586-023-06747-5

[27] Yuxuan Wan, Wenxuan Wang, Yiliu Yang, Youliang Yuan, Jen tse Huang, Pinjia He, Wenxiang Jiao, and Michael Lyu. 2024. LogicAsker: Evaluating and Improving the Logical Reasoning Ability of Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 2124–2155. doi:10.18653/v1/2024.emnlp-main.128

[28] Anjiang Wei, Yuheng Wu, Yingjia Wan, Tarun Suresh, Huanmi Tan, Zhanke Zhou, Sanmi Koyejo, Ke Wang, and Alex Aiken. 2025. SATBench: Benchmarking LLMs' Logical Reasoning via Automated Puzzle Generation from SAT Formulas. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (Eds.). Association for Computational Linguistics, Suzhou, China, 33832–33849. doi:10.18653/v1/2025.emnlp-main.1716

[29] Zihao Xu, Junchen Ding, Yiling Lou, Kun Zhang, Dong Gong, and Yuekang Li. 2025. Socrates or Smartypants: Testing Logic Reasoning Capabilities of Large Language Models with Logic Programming-based Test Oracles. arXiv:2504.12312 [cs.CL] https://arxiv.org/abs/2504.12312

[30] Zhuojie Yang, Wentao Wan, and Keze Wang. 2026. ORACLE: Optimizing Reasoning Abilities of Large Language Models via Constraint-Led Synthetic Data Elicitation. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 40 (2026), 34378–34386. doi:10.1609/aaai.v40i40.40735

[31] Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2024. MetaMath: Bootstrap Your Own Mathematical Questions for Large Language Models. In *The Twelfth International Conference on Learning Representations*. Vienna, Austria, 45040–45061. https://openreview.net/forum?id=N8N0hgNDRt

[32] Wanjun Zhong, Ruixiang Cui, Yiduo Guo, Yaobo Liang, Shuai Lu, Yanlin Wang, Amin Saied, Weizhu Chen, and Nan Duan. 2024. AGIEval: A Human-Centric Benchmark for Evaluating Foundation Models. In *Findings of the Association for Computational Linguistics: NAACL 2024*, Kevin Duh, Helena Gomez, and Steven Bethard (Eds.). Association for Computational Linguistics, Mexico City, Mexico, 2299–2314. doi:10.18653/v1/2024.findings-naacl.149

[33] Qin Zhu, Fei Huang, Runyu Peng, Keming Lu, Bowen Yu, Qinyuan Cheng, Xipeng Qiu, Xuanjing Huang, and Junyang Lin. 2025. AutoLogi: Automated Generation of Logic Puzzles for Evaluating Reasoning Abilities of Large Language Models. arXiv:2502.16906 [cs.CL] https://arxiv.org/abs/2502.16906

A Stage 1: Symbolic Proof DAG Synthesis

The main text (§3.1) states two properties of the proof DAGs: a topology fixes the shape of each proof before any formula is chosen, and a solver accepts every inference before a later inference may use it. This appendix describes the three components of Stage 1 that give these two properties: the rules that propose inferences (Appendix A.1), the topologies and rule sequences that fix the shape of a proof (Appendix A.2), and the solver checks that decide which inferences enter it (Appendix A.3). Appendix A.4 then follows the running example through all three components.

In Stage 1, we generated 5,000 solver-verified DAGs (3,800 PL and 1,200 FOL) from 5,050 attempts in a single run. Forward saturation is bounded, so every attempt ends, and an attempt that does not derive its target within the bounds is rejected. The released items are grounded in 4,951 of these DAGs, and the other 49 lose all of their natural language instances in later stages.

A.1 Forward Inference Rules

The rules propose and the solver decides. Each rule draws candidate inferences from the current proof state. Every candidate must then pass the solver checks of Appendix A.3 before it enters the proof state. The rules therefore determine which inference patterns a proof can contain, and the solver decides the validity of each step. Table 7 lists the 19 rules, where P, Q, R, S range over formulas, c is a ground constant, and $F(x)$ is the body of a universal formula. Universal Instantiation and Existential Generalization apply to formulas of first-order logic only, and the other rules apply to formulas of propositional logic and to ground formulas of first-order logic. Two of these rules, Contraposition and Material Implication (marked ‡), are restricted to formulas of propositional logic. In addition, Hypothetical Syllogism and Importation apply under a shared universal quantifier as well. In addition, some rules are restricted in scope or direction so that saturation stays bounded: goal-directed rules (marked †) only build sub-formulas of the target or of a seed, and De Morgan, Importation, and Material Implication are applied in one direction only, so saturation does not alternate between equivalent forms.

Lemma sharing rests on formula identity: commutative connectives are canonicalized by sorting their arguments, so a conclusion reached along different paths is stored as a single node, and a derived formula becomes a shared lemma whenever two later inferences use it.

A.2 Topologies and Rule Compatibility Graph

Each topology gives a proof a fixed shape before any formula is chosen: a linear topology gives depth, a fan-out reuses one lemma across several branches, and a fan-in joins two lines of argument. Table 8 lists the seven base topologies with the second topologies they compose with. A composition makes the proof of the base topology a sub-proof of the second topology. The target of the base topology enters the second topology as the shared lemma of a fan-out, the antecedent of an implication chain, or an input of a fan-in. The only exception is the composition of the first-order logic fan-out with the first-order logic fan-in, which passes one conjunct of the target to the fan-in. The seven base topologies and their 17 pairwise compositions therefore yield the 24 topologies of the corpus. Every base or composed topology that ends in a fan-out is built to apply a derived lemma in each of its branches. In the LogicDAG corpus, 1,270 of the 1,282 such DAGs reuse a derived lemma. In the remaining 12 DAGs, however, saturation derives each branch directly from the seeds. Lemma reuse is therefore measured on the extracted proof rather than assumed from the topology.

The topology does not fix the rules that fill the proof. In the propositional logic topologies, a rule sequence comes from a random walk over a rule compatibility graph (Table 9), whose edges connect a rule to the rules that accept the type of formula it derives. The arms of a dilemma are the only exception: each arm is a chain of Hypothetical Syllogism steps. Each step of the walk adds the seeds that its rule needs, so the seeds admit the sampled sequence as a forward proof of the target. The DAG, however, keeps the proof that forward saturation finds, which need not follow the sampled sequence: the walk fixes the seeds, and saturation fixes the proof.

The first-order logic topologies are instead built from fixed proof patterns in which one rule is drawn at random, and they ground their universals at a fresh constant through Universal Instantiation. The universal chain of first-order logic is the exception, since it links universal implications under the quantifier and stays in the universal form.

A.3 Solver Verification Statistics

The validity of each proof rests on the solver checks. During forward saturation, every proposed inference is verified before its conclusion enters the proof state. After proof extraction, each accepted DAG is verified as a whole: its seeds entail the target, and they do not entail its negation. An inference in propositional logic is accepted only when both Z3 and Vampire prove it, so a disagreement rejects it. An inference that involves a predicate or a quantifier, in contrast, is checked by Vampire alone, and a timeout or an inconclusive result rejects it. Before saturation, every seed set is also verified to be satisfiable, because an inconsistent seed set would entail any conclusion. Table 1 in §3.1 reports the checks of the run.

Z3 and Vampire agreed on all 2,166,196 checks in propositional logic, and every Vampire call ended with a proof or a counter-model within its time limit. The 50 rejected attempts either did not reach their target or produced a trivial or duplicate proof. In addition, no inference proposed by a forward rule was rejected, so the solvers confirmed every inference of the corpus and filtered out none. After generation, a separate post-check with its own formula parser re-verified all 37,345 inferences together with the entailment and the label of every DAG, and all 5,000 DAGs passed. Every inference and every label has thus been verified twice, once when it was derived and once from the stored formulas alone.

A.4 Illustrative Proof DAG

Figure 4 shows the proof DAG of the running example, a fan-out DAG in propositional logic whose items are all in the training split. Its proof has depth 3 and four inferences, and the derived formula A_{791} is a shared lemma. Saturation derived 39 formulas before it reached the target, and proof extraction kept only the four inferences on which the target depends. A Modus Ponens step derives the lemma from two seeds, and two further Modus Ponens steps then apply it in two branches that Conjunction Introduction

Table 7: The 19 forward inference rules, with their premises and conclusion. The Steps columns give the number of inferences that apply each rule in the 5,000 generated DAGs, and — marks a rule that does not apply to the fragment. † marks a rule that fires only toward the target or a seed, and ‡ marks a rule of propositional logic only.

			Steps	
Rule	**Premises**	**Conclusion**	**PL**	**FOL**
Propositional logic rules				
Modus Ponens	$P,\ P\to Q$	Q	5,027	2,665
Modus Tollens	$P\to Q,\ \neg Q$	$\neg P$	1,342	239
Hypothetical Syllogism	$P\to Q,\ Q\to R$	$P\to R$	4,932	2,980
	$\forall x(P\to Q),\ \forall x(Q\to R)$	$\forall x(P\to R)$	—	497
Disjunctive Syllogism	$P\vee Q,\ \neg P$	Q	4,037	205
Conjunction Introduction†	$P,\ Q$	$P\wedge Q$	1,674	405
Conjunction Elimination	$P\wedge Q$	P	129	82
Composition†	$P\to Q,\ P\to R$	$P\to(Q\wedge R)$	290	0
Double-Negation Elimination	$\neg\neg P$	P	815	0
Dilemma rules				
Constructive Dilemma	$P\to Q,\ R\to S,\ P\vee R$	$Q\vee S$	1,760	91
Destructive Dilemma	$P\to Q,\ R\to S,\ \neg Q\vee\neg S$	$\neg P\vee\neg R$	473	23
Bipolar Dilemma	$P\to Q,\ R\to S,\ P\vee\neg S$	$Q\vee\neg R$	848	15
Equivalence rules				
De Morgan	$\neg(P\wedge Q)$	$\neg P\vee\neg Q$	673	0
	$\neg(P\vee Q)$	$\neg P\wedge\neg Q$	132	0
Contraposition‡	$P\to Q$	$\neg Q\to\neg P$	1,232	—
Material Implication‡	$P\to Q$	$\neg P\vee Q$	226	—
Importation	$P\to(Q\to R)$	$(P\wedge Q)\to R$	426	0
	$\forall x(P\to(Q\to R))$	$\forall x((P\wedge Q)\to R)$	—	100
Biconditional Elimination (→)	$P\leftrightarrow Q,\ P$	Q	37	0
Biconditional Elimination (←)	$P\leftrightarrow Q,\ Q$	P	28	0
First-order logic rules				
Universal Instantiation	$\forall x\,F(x)$	$F(c)$	—	5,954
Existential Generalization†	$P(c)$	$\exists x\,P(x)$	—	8
Total			24,081	13,264

Table 8: The 24 proof topologies. Each of the seven base topologies is used alone and composed with every second topology in its row. As a second topology, the PL fan-in joins the first proof either by Disjunctive Syllogism (DS) or by Conjunction Introduction (CI).

Base topology	**Second topologies**
PL linear	PL linear, PL fan-out, PL fan-in (DS), PL fan-in (CI)
PL fan-out	PL linear, PL fan-out
PL fan-in	PL linear, PL fan-out
FOL linear	FOL linear, FOL fan-out, FOL fan-in
FOL fan-out	FOL linear, FOL fan-out, FOL fan-in
FOL fan-in	FOL linear, FOL fan-out, FOL fan-in
FOL universal chain	none

joins into the target. Both solvers verified every inference. Appendices B.2 and C.1 follow its instance in the Information Systems domain through Stages 2 and 3.

B Stage 2: Natural Language Grounding

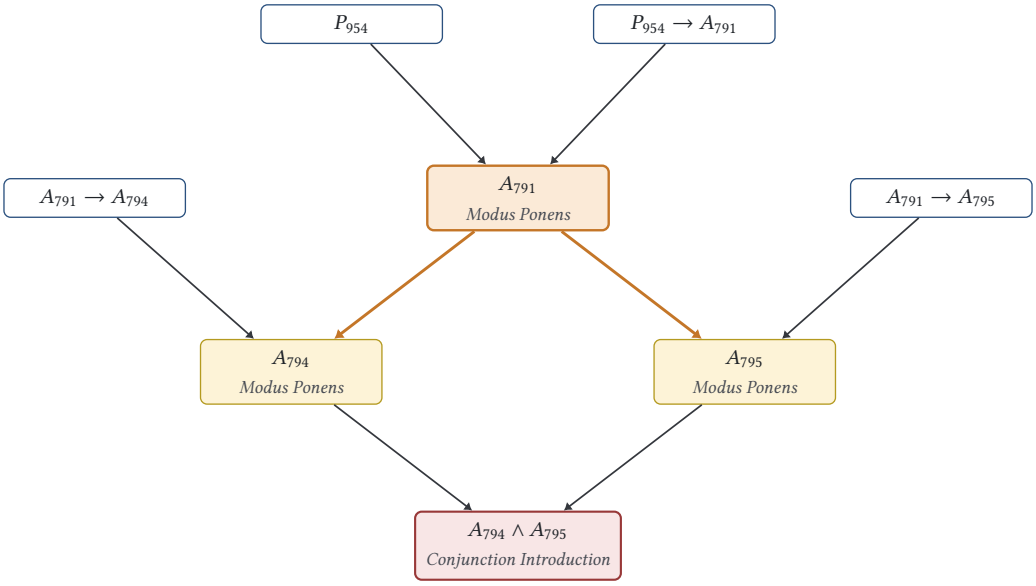
This appendix records what the model of Stage 2 receives, what it writes, and what is checked before an instance is admitted: the domains (Appendix B.1), the running example (Appendix B.2), the checks (Appendix B.3), and the prompts (Appendix B.4).

B.1 Domain Pool

A domain gives a proof its topic and leaves its logic unchanged. Stage 2 samples domains from a pool of 50 domains (Table 10), and the LLM chooses a scenario within the sampled domain that fits the proof. Each DAG is grounded in three distinct domains drawn uniformly at random, so the 5,000 DAGs yield 15,000 instances. The three instances of a DAG therefore share its proof and differ in their topic.

B.2 Illustrative Example

Figure 5 shows the Stage 2 output for the running example, a fan-out proof DAG in propositional logic of depth 3 from the training split (Figure 4), grounded in the Information Systems domain. It shows how Stage 2 splits the work: the model writes the atom map and the reasoning chain, and the template renderer writes every



Atom texts of the running example (Information Systems domain, training split)

P_{954}	The server receives the backup command.	A_{794}	The database snapshot is created.
A_{791}	The backup job completes successfully.	A_{795}	The integrity check passes.

Figure 4: Proof DAG of the running example, a fan-out DAG in propositional logic of depth 3. Blue nodes are seeds, yellow nodes are derived formulas, and the red node is the target. The orange node is the shared lemma A_{791}, which feeds two inferences. The table below the graph gives the text of each atom in the Information Systems domain.

Table 9: Rule compatibility graph of the propositional logic topologies: each rule, the type of formula it derives, and the rules that may follow it. The abbreviations are MP for Modus Ponens, MT for Modus Tollens, HS for Hypothetical Syllogism, DS for Disjunctive Syllogism, CT for Contraposition, MI for Material Implication, CI for Conjunction Introduction, CE for Conjunction Elimination, CO for Composition, IM for Importation, $\text{DM}_\wedge$ for De Morgan on $\neg(P \wedge Q)$, CD for Constructive Dilemma, DD for Destructive Dilemma, and BD for Bipolar Dilemma. * marks a start node, which seeds $\neg P \vee Q$ and P, and a successor "stop" means that the walk can end at this rule.

Node	Output type	Successors
Modus Ponens	atom	MP, DS, CI, IM, $\text{DM}_\wedge$
Disjunctive Syllogism	atom or negation	MP, MT, CI, IM, $\text{DM}_\wedge$
Modus Tollens	negation	MP, DS
Hypothetical Syllogism	implication	MT, HS, CT, MI, CO, CD, DD, BD
Contraposition	implication	MP, MT, HS, CT, MI
Composition	implication	MP, stop
Importation	implication	MP, stop
Material Implication	disjunction	DS, stop
Constructive Dilemma	disjunction	DS, stop
Destructive Dilemma	disjunction	DS, stop
Bipolar Dilemma	disjunction	DS, stop
De Morgan on $\neg(P \wedge Q)$	disjunction	DS, stop
De Morgan on $\neg(P \vee Q)$	conjunction	CE
Conjunction Introduction	conjunction	CE
Conjunction Elimination	conjunct	MP
Double-Negation Elimination	atom	MP
Biconditional Elimination	atom	MP
Seeded $\neg P \vee Q^*$	atom	MP, CI

Table 10: The 50 domains used for natural language grounding in Stage 2, in alphabetical order by row.

Domains	
Accounting	Agriculture
Animal Care	Animals & Nature
Anthropology	Archaeology
Art	Astronomy
Biology	Business & Organizations
Calendars & Scheduling	Chemistry
Computer Science	Cooking & Nutrition
Culture	Ecology
Economics	Education
Engineering	Environment
Everyday Life	Facilities Management
Film	Geography
Geology	History
Information Systems	Language & Grammar
Literature	Logistics
Manufacturing	Mathematics
Meteorology	Music
Navigation & Maps	Oceanography
Philosophy	Physics
Public Administration	Public Safety
Puzzles & Games	Renewable Energy
Social Life	Sociology
Space	Sports
Systems & Processes	Technology
Transportation	Urban Planning

premise and the conclusion from their formulas. Phase 2a maps each atom to a clause about a server's backup routine together with its natural negation. For example, $\neg A_{791}$ reads "The backup job does not complete successfully." Phase 2b rewrites the rule-labeled scaffold into one sentence per inference. In this reasoning chain, the shared lemma A_{791} is derived once and reused by two later steps. Phase 2c then admits the instance. The *True* item of the running example carries exactly these premises, this conclusion and this reasoning chain (Appendix C.1).

Knowledge framing. Knowledge framing varies the wording of conditional premises without changing the rules they state. The renderer states some conditional premises as the knowledge of a person in the scenario through a factive verb, as in "The system administrator knew that [...]", so each framed premise states the same rule as its plain form. The renderer decides deterministically which premises are framed, so the model does not choose them. In addition, the reading notes of Phase 2c (Figure 13) treat a framed premise as a rule of the scenario.

B.3 Faithfulness Check Statistics

An instance enters the corpus only after two kinds of check: deterministic checks on what the model writes in Phases 2a and 2b, and the faithfulness check of Phase 2c.

Deterministic checks. Because the renderer supplies all of the logic of a premise, an atom phrase must not carry logic of its own, such as a connective, a negation, or a hedge. In addition, an atom map must cover every atom. Likewise, a reasoning chain must contain exactly one sentence per inference, so each sentence can be traced to one verified step of the proof. An output that fails any of these checks is regenerated.

Faithfulness check. In Phase 2c, the same model, Qwen3.8-27B, is prompted as a verifier and judges the finished text as a whole. It compares the rendered premises and conclusion and the generated reasoning chain with the atom map and the rule-labeled scaffold (Figure 13). Only an instance that receives a True verdict is admitted. Its False verdicts remove 1,025 instances (6.8%) in which it finds the text unfaithful to the proof. In addition, 106 instances (0.7%) are removed because their verdict could not be parsed. Table 11 reports the outcome: 13,867 of the 15,000 instances (92.4%) are admitted, and the rejections fall mostly on instances in propositional logic.

Table 11: Outcome of the 15,000 Stage 2 instances (5,000 DAGs × 3 domains). Rejected instances are broken down by cause.

Outcome	PL	FOL	Total	%
Admitted, verdict True	10,294	3,573	13,867	92.4
Rejected	1,106	27	1,133	7.6
verdict False	1,000	25	1,025	6.8
unparseable output	106	0	106	0.7
generation error	0	2	2	<0.1
Total	11,400	3,600	15,000	100.0

B.4 Prompt Templates

Figures 6–13 reproduce verbatim the prompts sent in the run that built the LogicDAG corpus. They ask the model only for its share of the work. Phase 2a asks for the atom map, Phase 2b asks for one sentence per given step, and Phase 2c asks for a verdict with a one-sentence reason. All three phases call Qwen3.8-27B with its thinking mode disabled. In the templates, ⟨·⟩ marks a slot filled from the DAG, the sampled domain or an earlier phase, and a gray bracketed label marks the lines that are sent only under the stated condition.

C Stage 3: Task Framing

Stage 3 turns each admitted instance into labeled items: the construction fixes every label, and the LLM only writes and checks sentences, none of which can change a label. Appendix C.1 follows one instance through this stage and defines the constructions, and Appendix C.2 shows where the LLM enters.

C.1 Illustrative Example

Figure 2 shows the Stage 3 output for the running example of Appendices A.4 and B.2, a fan-out proof DAG in propositional logic grounded in the Information Systems domain. Its three items share the premises and the reasoning chain written in Stage 2. They differ in the conclusion, and the *False* and *Uncertain* items add a final step. The *Uncertain* item comes from the fresh-clause augmentation and asks about a statement that the premises do not mention. A model that reproduces the shared reasoning chain has therefore not yet answered any of the three items, because it must still judge what the chain settles about the conclusion of each item.

Phase 2a: Atom Mapping (Domain: Information Systems)

Atom		Phrase	Natural negation
A_{791}	↦	The backup job completes successfully.	The backup job does not complete successfully.
A_{794}	↦	The database snapshot is created.	The database snapshot is not created.
A_{795}	↦	The integrity check passes.	The integrity check does not pass.
P_{954}	↦	The server receives the backup command.	The server does not receive the backup command.

Person who knows the rules: The System Administrator **Proper names:** none (empty list)

Rendered Premises (template renderer, with the seed axiom on the left)

1. P_{954} The server receives the backup command
2. $P_{954} \rightarrow A_{791}$ If the server receives the backup command, then the backup job completes successfully
3. $A_{791} \rightarrow A_{794}$ If the backup job completes successfully, then the database snapshot is created
4. $A_{791} \rightarrow A_{795}$ If the backup job completes successfully, then the integrity check passes

Phase 2b: Reasoning Chain

(1) Since the server receives the backup command, and the stated rule indicates that receiving this command leads to a successful backup job, it follows that **the backup job completes successfully**. *[Modus Ponens on P_{954} and $P_{954} \rightarrow A_{791}$, giving A_{791}, the shared lemma, which is derived once here and reused in steps (2) and (3)]*

(2) Given that **the backup job completes successfully**, and the rule linking this completion to the creation of a database snapshot, we conclude that the database snapshot is created. *[Modus Ponens on A_{791} from step (1) and $A_{791} \rightarrow A_{794}$]*

(3) Since **the backup job completes successfully**, and the rule linking this completion to a passing integrity check, it follows that the integrity check passes. *[Modus Ponens on A_{791} from step (1) and $A_{791} \rightarrow A_{795}$]*

(4) Combining the established fact that the database snapshot is created with the fact that the integrity check passes, we conclude that the database snapshot is created and the integrity check passes. *[Conjunction Introduction on A_{794} from step (2) and A_{795} from step (3)]*

Rendered Conclusion ($A_{794} \wedge A_{795}$)

The database snapshot is created and the integrity check passes

Phase 2c: Faithfulness Check (the same model, prompted as a verifier)

Verdict: True

Reason: The natural language instantiation correctly maps all symbols, faithfully represents each Modus Ponens and Conjunction Introduction step, and states the correct conclusion.

Figure 5: Stage 2 output for the running example in the Information Systems domain: the atom map (Phase 2a), the rendered premises and target, the reasoning chain (Phase 2b), and the verdict (Phase 2c). The model writes only the atom map and the reasoning chain, and the renderer writes every premise and the conclusion. Bold marks the shared lemma A_{791}.

Phase 2a: Atom Mapping — System Prompt (Part 1: Instructions and Rules)

You are an expert in logical reasoning with a deep ability to ground abstract symbolic structures in concrete, self-contained scenarios. Given abstract symbolic atoms and information about the proof structure, assign a concrete English phrase to each atom so the full argument reads as a believable, coherent real-world scenario in the specified domain.

Rules:
1. Every phrase must be a simple, logically positive declarative clause (subject + verb phrase).
2. All atoms must belong to ONE coherent scenario within the given domain; choose a specific situation where the proof pattern reads naturally.
3. Treat the scenario as a self-contained micro-world. An implication A→B is a stipulated rule of that micro-world; never reverse it, strengthen it, or add outside knowledge or commonsense premises.
4. Conjunction target (A∧B): two facts that naturally co-occur in the same event.
5. Negation target (¬A): the direct, factual denial of A in the same scenario.
6. Disjunction target (A∨B) is INCLUSIVE: A, B, or both may hold. Never make the two disjuncts mutually exclusive unless the symbolic input explicitly says so.
7. Use ordinary, unambiguous concepts. Do NOT invent named individuals, specific organisations, or fictional events.
8. FOL predicate phrase must work as "entity [predicate]" grammatically. E.g., predicate name "soluble" with phrase "is soluble in water" → atom "soluble(x)" maps to "x is soluble in water.", atom "soluble(c0)" maps to "Salt is soluble in water." (when c0 = "salt"). See Example 3 for the full format.
9. FOL constant: use an unambiguous, well-known representative entity in the domain.
10. Atom phrases must be atomic propositions, not logical formulas. Do NOT use "if", "only if", "if and only if", "either", "implies", "equivalent", or "not the case" inside an atom phrase. The formal proof will add conditionals, disjunctions, conjunctions, and negations later.
11. Do not hide logical negation inside a positive atom (for example: "not", "never", "without", "unable", or "fails to"). Do not weaken or strengthen an atom with epistemic/frequency/necessity language such as "may", "might", "likely", "often", "must", or "necessarily".
12. Output ONLY valid JSON — no markdown fences, no text outside the JSON object.

Figure 6: System prompt of Phase 2a (atom mapping), part 1: instructions and rules.

Labels. Each admitted instance yields a *True* item, whose conclusion is the target, and a *False* item, whose conclusion is the negation of the target. Since the seed premises entail the target, both labels hold by construction. The negation is pushed inward to the literals and worded by the same templates as every *True* conclusion rather than by the LLM. A *False* conclusion is therefore worded in the

Phase 2a: Atom Mapping — System Prompt (Part 2: Examples)

== Example 1 — PL · linear_chain · Everyday Life ==
domain: Everyday Life
proof pattern: cascading cause-and-effect
atoms: ["P0", "P1", "P2", "P3"], target type: atom

```
{"domain": "Everyday Life", "atom_map": {
    "P0": "It is raining heavily outside.",
    "P1": "The road surface becomes slippery.",
    "P2": "The driver reduces speed.",
    "P3": "The car arrives at its destination late."
}}
```

== Example 2 — PL · modus_tollens_chain · Everyday Life ==
domain: Everyday Life
proof pattern: elimination by contradiction — ruling out hypotheses
atoms: ["P0", "P1", "P2"], target type: negation (¬P0)

```
{"domain": "Everyday Life", "atom_map": {
    "P0": "The power cable is connected.",
    "P1": "The battery is charged.",
    "P2": "The device turns on."
}}
```

== Example 3 — FOL · fol_chain · Physics ==
domain: Physics
proof pattern: universal property that propagates across all members of a category
atoms: ["metal(x)", "metal(c0)", "conducts(x)", "conducts(c0)"]
predicates: ["metal", "conducts"], constants: ["c0"], target type: atom

```
{"domain": "Physics",
  "predicate_map": {
     "metal": "is a metal",
     "conducts": "conducts electricity"
  },
  "constant_map": {"c0": "copper"},
  "atom_map": {
     "metal(x)": "x is a metal.",
     "metal(c0)": "Copper is a metal.",
     "conducts(x)": "x conducts electricity.",
     "conducts(c0)": "Copper conducts electricity."
  }}
```

The examples above illustrate the required format and level of coherence only. Ground your entire response in the provided domain and atoms — do not reuse content from the examples above.

Figure 7: System prompt of Phase 2a, part 2: few-shot examples.

same way as a *True* conclusion of the same form. The response of a *False* item is the *True* reasoning chain followed by one sentence that says why the target rules out the proposed conclusion.

An *Uncertain* item needs a conclusion that the premises neither entail nor refute, and Stage 3 draws it from two sources. First, a base *Uncertain* item takes its conclusion from the shape of the target. This conclusion is one disjunct of a disjunctive target, the converse of a conditional target, or the universal counterpart of an existential target, and the proof derives none of these. Its response appends to the *True* reasoning chain one or two sentences that identify the logical gap. Base *Uncertain* items therefore exist only for these three target shapes and are scarce. An instance whose target is an atom, a negation, or a conjunction has none. The target of the running example is the conjunction $A_{794} \wedge A_{795}$, so its *Uncertain* item comes from the fresh-clause augmentation.

Second, the fresh-clause augmentation adds one item to some of the instances without a base *Uncertain* item. The fresh clause C is a new statement that the LLM writes for the scenario, and the item asks about C or about its negation. Since C enters the formulas as a new atom, the label of the item follows from the formulas alone. In the *unmentioned* kind, the premises are unchanged and the label is *Uncertain*. In the *conditional* kind, one added premise joins C to a literal that the proof settles. Affirming the consequent and denying the antecedent then yield *Uncertain*, while modus ponens and modus tollens, which have the same surface form, yield *True* or *False*. Such an item is answered by checking which side of the added conditional the proof settles, and the mere appearance of C in a conditional does not decide the label. The response appends one templated step to the *True* reasoning chain. Together, the two sources give the 4,901 *Uncertain* items of the LogicDAG corpus, and 2,204 of them come from the augmentation. Z3 re-derives every label from the formulas released with the item.

C.2 Prompt Templates

The LLM enters Stage 3 only to write and check sentences (Figures 14–18). The same model as in Stage 2 writes the refutation sentence of each *False* item whose conclusion negates the target, the verdict sentences of the base *Uncertain* items, and the fresh clause of the augmentation. A second call then checks each fresh clause. Every label follows from the formulas, so none of these sentences can change it. Square-bracketed tokens denote fields filled at generation time.

Phase 2a: Atom Mapping — User Message Template

Domain: ⟨*domain*⟩
Proof pattern hint: this proof has the structure of '⟨*topology hint*⟩'.
→ Choose a scenario within this domain where the proof pattern feels most natural.

Fragment: ⟨*PL or FOL*⟩
Target type: ⟨*target type*⟩ | Target formula: ⟨*target formula*⟩
Atoms: ⟨*atoms, as a JSON list*⟩
Proof premises: ⟨*seed axioms, as a JSON list*⟩

[PL only]
Return JSON: {"domain": "<domain>", "atom_map": {atom: NL_phrase}}
[FOL only]
FOL predicates (assign property phrases): ⟨*predicate symbols*⟩
FOL constants (assign well-known entity names): ⟨*constant symbols*⟩

Return JSON: {"domain": "<domain>", "predicate_map": {pred: phrase}, "constant_map": {const: entity_name}, "atom_map": {atom_string: NL_phrase}}
FOL atom format: predicate "is soluble" + constant "salt" ⟶ {"soluble(c0)": "Salt dissolves in water.", "soluble(x)": "x dissolves in water."}
[always]

Also include these fields in the same JSON object:
- "negations": {atom: its natural negation} for EVERY atom listed above. Negate the verb the way a person would and keep every other word ("The valve is open." -> "The valve is not open."; "The pump starts." -> "The pump does not start."[FOL only:; "x conducts heat." -> "x does not conduct heat."]). Never write "it is not the case that".
[FOL only] - "entity_kind": "person" if the entities are people, otherwise "thing". When they are people, a constant may be a common first name (e.g. "Maya"); this relaxes rule 7 for constants only.
- "agent": a short name or role of one person in this scenario who knows its rules (e.g. "Leo" or "The site engineer").
- "names": every proper name used in your sentences (people, places, organisations, titles), written exactly as in the sentences, e.g. ["Maya", "Citizen Kane"]; [] if there are none. Do not list common nouns (e.g. "sandstone", "the pump").

Appended on a retry after an invalid atom map:
Previous attempt was invalid: ⟨*reasons*⟩
Rewrite the atom_map only. Each atom must be a simple positive declarative clause.[if only phrases collide: No two atoms may share the same phrase.] Do not put conditionals, disjunctions, biconditionals, negations, lexical negation, epistemic hedges, necessity language, or symbolic logic in any atom phrase.[if a negation is invalid: Keep the other fields, and give "negations" for every atom: the same clause with only its verb negated.]

Figure 8: User message template of Phase 2a, with the text appended on a retry.

D Stage 4: Quality Gate

The solvers verify the formulas of every item, but not the sentences that Stages 2 and 3 write for them, so Stage 4 checks those sentences wherever a fixed reference decides the outcome. Stage 4 releases an instance only if all of its items pass four groups of checks, and each group is decided against a fixed reference. First, the *structure* checks compare each reasoning chain with the length its proof requires. Second, the *rendering* checks compare the wording with the atom map and reject malformed negations and formula symbols left in the text. Third, the *reasoning* checks compare the conditional statements of a reasoning chain with the proof and reject a *False* or *Uncertain* reasoning chain that claims the conclusion follows. Finally, the *arithmetic* checks evaluate the arithmetic statements of the text. All four groups rely on formal references, so the gate does not judge whether a sentence is plausible. As a result, a flaw that only the meaning of the text reveals can pass the gate. An instance that fails any check is removed with all of its items. Table 2 reports how many instances and items the gate removes.

Phase 2b: Reasoning Chain — System Prompt (Part 1: Instructions and Rules)

You are an expert in logical reasoning, skilled at transforming formal proof steps into fluent, coherent natural-language explanations. Given a proof scaffold — a step-by-step logical derivation in semi-natural language — rewrite it as a fluent, connected reasoning trace. Each output sentence must correspond to exactly one scaffold step, in the same order.

Rules:
1. Use the scaffold as a logical blueprint; do NOT copy it verbatim.
2. Vary connectives to match the logical move being performed:
   - Modus Ponens (MP): "Since [P] holds, and if [P] then [Q], it follows that [Q]."
   - Hypothetical Syllogism (HS): "Chaining [P→Q] with [Q→R], the combined implication gives [P→R]."
   - Modus Tollens (MT): "[Q] did not occur, and if [P] then [Q], so [P] cannot hold."
   - Conjunction Introduction: "Combining [P] and [Q], we establish [P∧Q]."
   - Conjunction Elimination: "From the conjunction [P∧Q] we extract [P]." (applies to AND, never OR)
   - Disjunctive Syllogism (DS): "Since [P] is ruled out, the only remaining option from [P∨Q] is [Q]."
   - Constructive Dilemma (CD): "Whether [P] or [R] holds — [P] leads to [Q], [R] leads to [S] — so in either case [Q∨S]."
   - Destructive Dilemma (DD): "Since at least one of [Q] or [S] fails (i.e., [¬Q∨¬S]), by contrapositive at least one of [P] or [R] also fails, giving [¬P∨¬R]."
   - Bipolar Dilemma (BD): "Given [P∨¬S]: if [P] holds then [Q] follows (via [P→Q]); if [¬S] holds then [¬R] follows (contrapositive of [R→S]); in either case [Q∨¬R]."
   - Contraposition: "By contrapositive of [P→Q]: if [¬Q] then [¬P]."
   - Material Implication: "The conditional [P→Q] is equivalent to [¬P∨Q] (or vice versa)."
   - Biconditional Elim (Forward): "Since [P↔Q] and [P] holds, by the forward direction we conclude [Q]."
   - Biconditional Elim (Backward):"Since [P↔Q] and [Q] holds, by the backward direction we conclude [P]."
   - Universal Instantiation (UI): "Applying the universal rule [∀x.F(x)] to the specific individual [c], we get [F(c)]."
   - Existential Generalization: "Since [c] satisfies [F(c)], there exists at least one entity with that property."
   - Double Negation: "Removing the double negation [¬¬P] gives [P]."
3. Each sentence must be self-contained and unambiguous.
4. Do not skip, merge, or add steps beyond the scaffold.
5. For FOL, Universal Instantiation and Modus Ponens are separate proof steps:
   - Universal Instantiation derives a specific conditional from a universal rule.
   - Modus Ponens then uses that specific conditional plus a fact to derive the next fact.
   Never merge those two moves into one sentence.
6. The final sentence must clearly assert the conclusion.
7. Preserve the exact entity, predicate meaning, and polarity in every scaffold step. Do not substitute one entity for another or turn a positive fact into a negative fact (or vice versa).
8. Use only information in the scaffold. Do not add outside knowledge, a causal bridge, a new premise, or modal/hedging language that changes assertion strength.
9. Every disjunction is inclusive unless the scaffold explicitly states otherwise; do not paraphrase OR as "exactly one" or "but not both".
10. Output ONLY valid JSON with key "proof_trace_nl" (list of strings, one per step).

Figure 9: System prompt of Phase 2b (reasoning chain), part 1: instructions and rules.

Phase 2b: Reasoning Chain — System Prompt (Part 2: Examples 1–3)

== Example 1 — PL · linear_chain · 3 steps · Everyday Life ==
Scaffold:
Step 1 (Modus Ponens): "It is raining heavily outside." + "If it is raining heavily outside, the road surface becomes slippery." ⟶ "The road surface becomes slippery."
Step 2 (Modus Ponens): "The road surface becomes slippery." + "If the road surface becomes slippery, the driver reduces speed." ⟶ "The driver reduces speed."
Step 3 (Modus Ponens): "The driver reduces speed." + "If the driver reduces speed, the car arrives at its destination late." ⟶ "The car arrives at its destination late."

{"proof_trace_nl": [
"It is raining heavily, and the stated rule says that heavy rain makes the road slippery, so the road is slippery.",
"The road is slippery, and the stated rule links a slippery road to reduced speed, so the driver reduces speed.",
"The driver has reduced speed, and the stated rule links reduced speed to late arrival, so the car arrives late."
]}

== Example 2 — PL · modus_tollens_chain · 2 steps · Everyday Life ==
Scaffold:
Step 1 (Modus Tollens): "If the battery is charged, the device turns on." + "It is not the case that the device turns on." ⟶ "It is not the case that the battery is charged."
Step 2 (Modus Tollens): "If the power cable is connected, the battery is charged." + "It is not the case that the battery is charged." ⟶ "It is not the case that the power cable is connected."

{"proof_trace_nl": [
"A charged battery is sufficient for the device to turn on, yet the device did not turn on — so the battery cannot be charged.",
"A connected power cable would have charged the battery, but the battery is not charged, so the power cable is not connected."
]}

== Example 3 — FOL · fol_chain · 2 steps · Biology ==
Scaffold:
Step 1 (Universal Instantiation): "for all x, If x is a plant, x needs water" ⟶ "If fern is a plant, fern needs water"
Step 2 (Modus Ponens): "Fern is a plant." + "If fern is a plant, fern needs water" ⟶ "Fern needs water."

{"proof_trace_nl": [
"Applying the universal rule to the specific case of fern gives the conditional: if fern is a plant, then fern needs water.",
"Since fern is a plant and that specific conditional applies to fern, it follows that fern needs water."
]}

Figure 10: System prompt of Phase 2b, part 2: few-shot examples.

Phase 2b: Reasoning Chain — System Prompt (Part 3: Examples 3b–5)

== Example 3b — FOL · fol_existential · 3 steps · Ecology ==
Scaffold:
Step 1 (Universal Instantiation): "for all x, If x is a migratory bird, x crosses the equator" → "If the Arctic tern is a migratory bird, the Arctic tern crosses the equator"
Step 2 (Modus Ponens): "The Arctic tern is a migratory bird." + "If the Arctic tern is a migratory bird, the Arctic tern crosses the equator" → "The Arctic tern crosses the equator."
Step 3 (Existential Generalization): "The Arctic tern crosses the equator." → "There is at least one bird that crosses the equator."

{"proof_trace_nl": [
"Applying the universal rule to the Arctic tern gives the conditional: if the Arctic tern is a migratory bird, then it crosses the equator.",
"The Arctic tern is indeed a migratory bird, and that conditional applies directly to it, so we conclude that the Arctic tern crosses the equator.",
"Having established that the Arctic tern crosses the equator, we now know that at least one bird makes this crossing — the existence of such a bird is confirmed."
]}

== Example 4 — PL · constructive_dilemma_intro + disj_syllogism · 2 steps · Everyday Life ==
Scaffold:
Step 1 (Constructive Dilemma): "If the storm hits, the power goes out." + "If the cable is cut, the internet disconnects." + "Either the storm is hitting or the cable has been cut." → "The power goes out or the internet disconnects."
Step 2 (Disjunctive Syllogism): "The power goes out or the internet disconnects." + "It is not the case that the power goes out." → "The internet disconnects."

{"proof_trace_nl": [
"Whether a storm strikes or the cable is cut — in the first case the power fails, in the second the internet drops — either way at least one of those consequences must follow: the power goes out or the internet disconnects.",
"Since the power has not gone out, that possibility is eliminated; by disjunctive syllogism the only remaining conclusion is that the internet has disconnected."
]}

== Example 5 — PL · biconditional_chain · 2 steps · Everyday Life ==
Scaffold:
Step 1 (Biconditional Elimination (Forward)): "The alarm sounds if and only if there is smoke." + "The alarm is sounding." → "There is smoke."
Step 2 (Modus Ponens): "There is smoke." + "If there is smoke, the sprinkler activates." → "The sprinkler activates."

{"proof_trace_nl": [
"Since the alarm sounds if and only if there is smoke, and the alarm is currently sounding, we can conclude that smoke is present.",
"Smoke being present, together with the rule that smoke triggers the sprinkler, establishes that the sprinkler has activated."
]}

The examples above illustrate the required style and step-count only. Write sentences that are grounded solely in the provided scaffold content — do not reuse phrases from the examples above.

Figure 11: System prompt of Phase 2b, part 3: further few-shot examples and the closing instruction.

Phase 2b: Reasoning Chain — User Message Template

Premises:
⟨ *verbalized premises, as a JSON list* ⟩

Conclusion:
⟨ *verbalized target, as a JSON string* ⟩

Expected number of trace sentences: ⟨ n ⟩
[if the proof uses Universal Instantiation]

FOL note: keep Universal Instantiation as its own sentence. Do not combine it with a later Modus Ponens sentence.
[if a Modus Ponens step uses a conditional as its minor premise]

Conditional-premise note: in step⟨ s ⟩ ⟨ *step numbers* ⟩, the fact that Modus Ponens uses is itself a conditional ("if A, then ..."). Cite that conditional as the reason. Do not write that A holds: neither the premises nor any earlier step establish it.
[if the proof has a Destructive or Bipolar Dilemma step]

Dilemma note: in step⟨ s ⟩ ⟨ *step numbers* ⟩, argue by cases inside the one sentence: "if <first case> holds, then <what follows>; if <second case> holds, then <what follows>; so in either case <conclusion>". For a Destructive Dilemma the two cases are the two failed results, and each gives its failed cause by contraposition.
[if the last step derives a conditional target by Hypothetical Syllogism, Importation, Composition or Contraposition]

Conditional-proof note: the conclusion is a conditional. Write the last step (step ⟨ n ⟩) by supposition, still as one sentence: begin with "Suppose" and the conclusion's if-part (for a statement about anyone or anything, suppose one arbitrary individual has that property), show that its then-part follows, and end by stating the conclusion.
[always]
Proof scaffold (follow exactly, one sentence per step):
Step ⟨ i ⟩ (⟨ *rule name* ⟩): "⟨ *premise* ⟩" + "⟨ *premise* ⟩" → "⟨ *conclusion* ⟩" [one line per inference, with its one to three premises joined by +]

Rewrite as a fluent reasoning trace with exactly ⟨ n ⟩ sentences.

Appended on a retry after a wrong number of sentences:
Previous attempt had ⟨ k ⟩ sentences, but the scaffold has ⟨ n ⟩ steps. Rewrite again with EXACTLY ⟨ n ⟩ sentences. Do not merge Universal Instantiation with Modus Ponens.

Appended on a retry after symbolic tokens:
Previous attempt contained symbolic identifiers (atom codes, predicate codes, or logic operators such as ∀ ∃ ∧ ∨ ¬ → ↔). Replace EVERY symbolic token with its English phrase from the scaffold. Your output must be entirely natural English — no code-like tokens at all.

Figure 12: User message template of Phase 2b, with the text appended on a retry. Each note is added only when the proof contains the corresponding step.

Phase 2c: Faithfulness Check — System Prompt

You are an expert in formal logic and natural language.
You will be given a symbolic proof and its natural language (NL) instantiation.
Your task is to verify whether the NL faithfully and correctly represents the symbolic proof.

Check ALL of the following:
1. Atom mapping consistency — each symbol maps to the same NL phrase throughout premises, trace, and conclusion.
2. Inference faithfulness — each NL step correctly expresses the stated logical rule (Modus Ponens, Modus Tollens, etc.) using the correct premises and conclusion.
3. Conclusion correctness — the NL conclusion correctly expresses the symbolic target formula.

Output ONLY valid JSON with two keys:
"verdict": one of "True", "False", or "Uncertain"
"reason": one short sentence identifying the specific issue (if any)

"True" — the NL faithfully represents the symbolic proof in all respects.
"False" — the NL contains at least one error (wrong mapping, wrong inference step, wrong conclusion).
"Uncertain" — cannot determine faithfulness from the given information.

Reading notes:
- A premise such as "Leo knew that if A, then B" states the rule "if A, then B" as a given fact of the scenario.
- "Anyone who P Q" and "Anything that P Q" state the universal rule "for every x, if P(x) then Q(x)"; "There is someone who P" states that some x has P.
- "Everyone who P Q", "Every person who P Q", "Everything that P Q", "Whoever P Q", "Whatever P Q", "If someone P, then that person Q" and "If something P, then that thing Q" state the same universal rule; "Someone P", "Something P" and "At least one person P" state that some x has P.
- A step may argue by supposition ("Suppose A; ... so if A, then C"); that correctly states the derived conditional.
- A dilemma step may be written as two explicit cases joined by "in either case".

Phase 2c: Faithfulness Check — User Message Template

== Symbolic atom mapping ==
⟨*symbol*⟩: "⟨*phrase*⟩" [one line per atom and per negated atom, sorted by symbol]

== Symbolic proof (with rules) ==
⟨*scaffold, as in the Phase 2b message*⟩

== NL instantiation to verify ==
Premises:
1. ⟨*premise 1*⟩ [one numbered line per premise]

Reasoning trace:
Step 1: ⟨*trace sentence 1*⟩ [one line per trace sentence]

Conclusion: ⟨*verbalized target*⟩

Does the NL faithfully represent the symbolic proof? Check mapping consistency, inference faithfulness, and conclusion correctness.

Figure 13: System prompt and user message template of Phase 2c (faithfulness check). Only a True verdict admits the instance.

False Refutation and Uncertain Verdict Sentences — Shared System Prompt

You are a logical-reasoning annotation assistant. Your task is to write concise natural-language verdict explanations for natural language inference (NLI) examples, in which a set of premises either entails, refutes, or neither entails nor refutes a proposed conclusion.

Rules:
1. Do not fabricate support or evidence for any conclusion not entailed by the given premises.
2. For a False verdict: explain why the proposed conclusion is ruled out by what the premises entail.
3. For an Uncertain verdict: explain why the proposed conclusion is not determined by the premises (neither entailed nor refuted under an open-world assumption).
4. Use only information provided in the prompt; do not draw on outside knowledge.
5. Respond with a single valid JSON object and nothing else — no preamble, markdown fences, or surrounding text.

Figure 14: Shared system prompt for the *False* refutation sentence and the *Uncertain* verdict sentence of Stage 3.

False Refutation Sentence — User Template

Natural language inference — False verdict annotation.

Premises:
[PREMISES]

Proof that the premises entail the true conclusion (complete — do not extend or continue these steps):
[TRUE_PROOF_TRACE]

True conclusion (entailed by the premises): "[TRUE_CONCLUSION]"
Proposed conclusion (verdict: False): "[FALSE_CONCLUSION]"

Write one refutation sentence that serves as the verdict explanation. It must be a standalone verdict statement, not a continuation of the proof steps above.

Requirements:
- State that the proposed conclusion is false or ruled out by the premises.
- Ground the refutation in what the premises entail: "[TRUE_CONCLUSION]" contradicts the proposed conclusion.
- Do not write a proof step, derivation, or inference.
- Do not assert that the proposed conclusion follows or is consistent with the premises.
- Do not introduce information beyond the given premises.
- Keep it concise (at most 35 words).

Respond with JSON: {"sentence": "<one sentence>"}

Figure 15: User prompt for the refutation sentence of a *False* item.

Uncertain Verdict Sentence — User Template

Natural language inference — Logical independence annotation.

Premises:
[PREMISES]

Established conclusion: "[TRUE_CONCLUSION]"
Proposed conclusion (verdict: Uncertain): "[UNCERTAIN_CONCLUSION]"

[CONTEXT_HINT]

Write one to two objective deductive sentences explaining why the proposed conclusion does not follow.

Strict Requirements:
- State clearly and objectively that the premises do not entail or determine the proposed conclusion.
- Reference the established conclusion and explain specifically the logical gap that prevents it from determining the proposed conclusion.
- Do NOT use conversational hesitation, hedging, or speculative boilerplate (DO NOT begin sentences with 'However,', and DO NOT write 'cannot be determined', 'insufficient information', 'we cannot know', or 'not established').
- Express the reasoning gap in your own objective words (e.g. 'The premises establish that …, but leave … logically independent.').
- Do not assert that the proposed conclusion is false or impossible.
- Do not introduce information beyond the given premises.

Respond with JSON: {"trace": ["<sentence 1>", "<sentence 2 (optional)>"]}

Appended on a retry:
CRITICAL CONSTRAINT VIOLATION: Your previous response contained prohibited hedging pattern '[PATTERN]'. Re-write without any conversational hesitation or hedging words ('However,', 'cannot be determined', etc.). State the formal logical independence directly.

Figure 16: User prompt for the verdict sentence of a base *Uncertain* item. The context hint takes one of the variants of Figure 17.

Uncertain Verdict Sentence — Context Hints (three variants for `[CONTEXT_HINT]`)

One disjunct of an $A \vee B$ target:
The premises entail the disjunction: "[TRUE_CONCLUSION]". While this confirms that at least one of the two constituent propositions holds, it does not determine which specific one holds. The proposed conclusion asserts one constituent individually, which is neither entailed nor refuted by the disjunction alone.

Universal counterpart of an $\exists x.\, P(x)$ target:
The premises entail: "[TRUE_CONCLUSION]" — an existential statement asserting that at least one entity satisfies a property. The proposed conclusion asserts the universal generalisation, which is a strictly stronger claim not determined by the existential.

Converse of a $P \rightarrow Q$ target:
The premises entail: "[TRUE_CONCLUSION]" — a conditional statement (if P then Q). The proposed conclusion is the converse (if Q then P), which is a logically independent claim: the truth of $P \rightarrow Q$ does not entail $Q \rightarrow P$.

Figure 17: The three context hints substituted into the context-hint field of Figure 16, selected by the kind of the base *Uncertain* item.

Fresh Clause Generation and Check — System Prompt

You are a careful writer of natural-language logic exercises. You add one new statement to an existing exercise. Respond with a single valid JSON object and nothing else — no preamble, markdown fences, or surrounding text.

Fresh Clause Generation — User Template

Below are the premises of a logic exercise in the domain "[DOMAIN]".

Premises:
[PREMISES]

Statements already used in this exercise (do not restate or paraphrase them):
[ATOM_TEXTS]

Write ONE new statement about the same scenario that the premises neither state, deny nor make obvious: a different fact, event or property a reader might wonder about, in the same register and tense as the premises. Then write its natural negation.

Requirements:
- One short declarative sentence of at most 20 words, about the same subjects or setting.
- No connectives: do not use "if", "then", "or", "and", "either", "unless", "both".
- No numbers.
- It must not follow from the premises and must not contradict them.
- It must not be a near-synonym or a consequence of any statement listed above.
- The negation is the same sentence with "not" placed in its verb group ("does not", "is not", "did not"), nothing else changed.

Respond with JSON: {"clause": "<sentence>", "negation": "<negated sentence>"}

Appended on a retry after a failed check:
Your previous statement was rejected: [REASONS]. Write a different statement that meets every requirement.

Appended on a retry after a "settled" verdict:
Your previous statement was rejected because the premises already settle it. Write a statement the premises leave genuinely open.

Fresh Clause Check — User Template

Premises:
[PREMISES]

Statement: "[CLAUSE]"

Would a careful reader of these premises consider the statement settled — true, false, or obvious given what the premises say, including anything they clearly presuppose? Answer "yes" if it is settled in any of these ways, and "no" only if the premises leave it genuinely open.

Respond with JSON: {"settled": "yes" or "no", "reason": "<one sentence>"}

Figure 18: Prompts for generating and checking the fresh clause of the augmentation. Only a clause that the check judges open is used.

E Experiment Details

This appendix gives the details behind Section 4: the statistics of the LogicDAG corpus (Appendix E.1), the fine-tuning setup (Appendix E.2), token usage in training (Appendix E.3), the held-out splits (Appendix E.4), qualitative examples (Appendix E.5), and the evaluation protocol (Appendix E.6).

E.1 Corpus Statistics

Table 12 reports the size and labels of each split. Across the splits, the 33,126 released items are grounded in 4,951 proof DAGs. *True* and *False* items are nearly balanced because every admitted instance yields one of each. *Uncertain* items (14.8%), in contrast, come only from three target shapes and from the fresh-clause augmentation (Appendix C.1).

Tables 3 and 13 report the depth and width of the proof DAGs, where depth is the number of inferences on the longest path from a seed to the target and width is the largest number of proof nodes at the same depth. Most proofs are three to five inferences deep and draw on several premises at once, and only proofs in propositional logic go beyond depth eight. A DAG reuses a derived lemma when a derived node is a premise of at least two inferences, and 1,311 of the 4,951 DAGs (26.5%) do so. Lemma reuse thus occurs in about a quarter of the proof DAGs.

Table 14 reports how many DAGs apply each of the 19 forward inference rules. Every rule occurs in the corpus, but the usage is uneven: modus ponens and hypothetical syllogism occur in most proofs, while the two biconditional eliminations and existential generalization each occur in fewer than one DAG in a hundred.

Table 12: Size and labels of the three splits, which share no proof DAG.

Split	DAGs	Instances	Items	True	False	Uncertain
Train	4,705	12,991	31,495	13,431	13,404	4,660
Validation	122	326	785	340	334	111
Test	124	351	846	354	362	130
Total	4,951	13,668	33,126	14,125	14,100	4,901

Table 13: Proof width of the released proof DAGs, by logical fragment.

Width	PL	FOL	%	Width	PL	FOL	%
1	20	0	0.4	10	189	71	5.3
2	76	145	4.5	11	186	50	4.8
3	274	121	8.0	12	57	44	2.0
4	469	145	12.4	13	69	32	2.0
5	595	193	15.9	14	22	15	0.7
6	521	91	12.4	15	25	6	0.6
7	534	95	12.7	16	14	4	0.4
8	374	97	9.5	17	2	4	0.1
9	322	86	8.2	18	3	0	0.1

Table 14: Released proof DAGs whose proof applies each inference rule, by logical fragment.

Rule	PL	FOL	All	%
Modus Ponens	2,794	808	3,602	72.8
Hypothetical Syllogism	2,335	982	3,317	67.0
Disjunctive Syllogism	2,325	168	2,493	50.4
Conjunction Introduction	1,340	379	1,719	34.7
Constructive Dilemma	1,317	69	1,386	28.0
Modus Tollens	1,043	89	1,132	22.9
Universal Instantiation	0	1,100	1,100	22.2
Contraposition	867	0	867	17.5
Double-Negation Elimination	761	0	761	15.4
De Morgan	729	0	729	14.7
Bipolar Dilemma	653	15	668	13.5
Importation	405	99	504	10.2
Destructive Dilemma	363	23	386	7.8
Composition	280	0	280	5.7
Material Implication	212	0	212	4.3
Conjunction Elimination	129	41	170	3.4
Biconditional Elimination ($\rightarrow$)	37	0	37	0.7
Biconditional Elimination ($\leftarrow$)	28	0	28	0.6
Existential Generalization	0	8	8	0.2

E.2 Fine-tuning Setup

All three models receive full-parameter supervised fine-tuning (SFT) on the training split with one shared configuration (Table 15) on a single NVIDIA H200 GPU. Each item is a three-message conversation (system, user, and assistant) formatted with the chat template of the base model, and the loss is computed on the assistant tokens only. The system message of every training item is "You are a helpful AI assistant.", the same system message used at evaluation (Appendix E.6). Each model is trained for three epochs.

Table 15: Shared training hyperparameters of the three fine-tuning runs, one run per model. All three runs use the same values.

Hyperparameter	Value
Fine-tuning type	Full-parameter supervised fine-tuning
Hardware	1× NVIDIA H200 NVL (141 GB)
Precision	bfloat16
Optimizer	AdamW, $\beta = (0.9, 0.95)$
Learning rate	1×10^{-6}
LR scheduler	Cosine
Warmup ratio	0.03
Weight decay	0.01
Gradient clipping	Max. norm 1.0
Global batch size	128
Micro batch size per GPU	4
Max sequence length	8,192 tokens
Epochs	3
Loss	Assistant tokens only
Gradient checkpointing	Enabled
Random seed	1

E.3 Token Usage

Table 16 reports the length of the released items under the Llama-3 tokenizer, with the chat template applied as in training. The prompt of an item is masked from the loss. Its response carries the loss and holds the reasoning chain, the final verdict and answer, and the end-of-turn tokens. The longest item has 2,569 tokens, which is below the maximum sequence length of 8,192 tokens, so every training sequence is kept in full. Since each step of the shared reasoning chain restates one verified inference, the length of a response grows with the size of its proof, from 203 loss tokens on average at depth 2 to 691 at depth 6.

Table 16: Token counts of the released splits under the Llama-3 tokenizer, which the three models share. Response tokens carry the loss, and Total gives the response tokens of each split.

Split / statistic	Prompt	Response
Train (31,495 items)		
Mean	288	407
Median	280	353
Max	617	2,025
Total	–	12,822,908
Validation (785 items)		
Mean	297	442
Median	295	406
Max	491	1,244
Total	–	347,043
Test (846 items)		
Mean	282	380
Median	274	336
Max	546	992
Total	–	321,242

E.4 LogicDAG as an Evaluation Set

The held-out splits can also serve as a test of deduction, since every label follows from a verified proof and no proof DAG of a held-out item occurs in training. The validation and test splits hold 785 and 846 items from 122 and 124 proof DAGs, respectively, and both cover all 24 topologies and all 50 domains. Tables 17 and 18 break down their items by label and construction, and by logical fragment, proof structure, and depth.

Table 17: Labels of the validation and test items by construction. MP, MT, AC, and DA denote the inference pattern of a fresh-clause item: modus ponens, modus tollens, affirming the consequent, and denying the antecedent.

	Validation		Test	
Label / construction	**Count**	**%**	**Count**	**%**
True	340	43.3	354	41.8
Target	326		351	
Fresh clause, MP	8		2	
Fresh clause, MT	6		1	
False	334	42.5	362	42.8
Negated target	326		351	
Fresh clause, MP	5		4	
Fresh clause, MT	3		7	
Uncertain	111	14.1	130	15.4
One disjunct of the target	43		55	
Converse of the target	15		25	
Fresh clause, unmentioned	29		33	
Fresh clause, AC	14		9	
Fresh clause, DA	10		8	
Total	785	100.0	846	100.0

Table 18: Validation and test items by logical fragment, proof structure, and proof depth.

	Validation		Test	
Category	**Count**	**%**	**Count**	**%**
Fragment				
Propositional logic (PL)	590	75.2	635	75.1
First-order logic (FOL)	195	24.8	211	24.9
Structure				
Linear	275	35.0	296	35.0
Fan-out	65	8.3	68	8.0
Fan-in	80	10.2	100	11.8
Universal chain	18	2.3	15	1.8
Composition	347	44.2	367	43.4
Depth				
2	127	16.2	178	21.0
3	165	21.0	191	22.6
4	193	24.6	234	27.7
5	164	20.9	148	17.5
6–8	136	17.3	95	11.2
Total	785	100.0	846	100.0

E.5 Qualitative Examples

Figures 19 and 20 compare the responses of Llama-3.1-8B-Instruct before and after fine-tuning on LogicDAG on two Multi-LogiEval items in propositional logic. In the first item, a nested conditional states that if the head baker uses high-quality flour, then if she kneads the dough thoroughly, the bread rises perfectly and has an airy texture. The question then states that she kneaded the dough thoroughly but the bread did not rise perfectly. Both models answer No. The fine-tuned model applies the rule to the given kneading and then applies modus tollens, in 189 tokens. The base model, in

contrast, does not use the rule. It cites outside factors such as yeast quality and temperature, concludes only that the use of high-quality flour cannot be established, and uses 251 tokens. In the second item, a disjunction states that Mark finished his errands or Stella was not nervous and excited. The fine-tuned model supposes that it is Stella's first day and derives that she is nervous and excited, so the second disjunct is false. It then infers by disjunctive syllogism that Mark finished his errands and concludes by modus ponens that he has free time. The base model misreads the disjunction as the conditional that Mark did not finish his errands if Stella was nervous and excited, and then denies the antecedent of the conditional that finishing his errands gives him free time.

E.6 Evaluation Prompt

Base models and fine-tuned models are posed the same questions and scored by the same rule. Every benchmark is evaluated zero-shot with chain-of-thought prompting and greedy decoding in vLLM [12], with the chat template and tokenizer of the base model. For LogicBench, Multi-LogiEval, FOLIO, LogiQA2.0, and AGIEval LSAT-LR, the system message is that of the training items ("You are a helpful AI assistant."). Figure 21 shows the user messages of these five benchmarks. The generation budget is 1,024 new tokens for LogicBench, FOLIO, and Multi-LogiEval, and it is 2,048 new tokens for LogiQA2.0 and AGIEval LSAT-LR. LogicAsker [27] is posed with the chain-of-thought prefix of its authors. GSM8K, MATH-500, and AGIEval-MATH are posed with the chain-of-thought input of the AGIEval data loader and an instruction to box the final answer. These four benchmarks use the same system message.

Scoring. The answer is read from the last boxed expression of a response, with fallbacks for an explicit answer statement. On AGIEval LSAT-LR, where many responses state an option without the requested box, the reader also accepts an option stated in another form. A response without a readable answer is scored as incorrect. LogicAsker is scored with a flexible read of a closing Yes or No. GSM8K and MATH-500 are scored with Math-Verify[2], and AGIEval-MATH is scored with the grader of AGIEval. In Table 4, the Multi-LogiEval scores pool depths 1–5, and the FOLIO scores come from the validation set of FOLIO v2.

[2] https://github.com/huggingface/Math-Verify

Multi-LogiEval, propositional logic, depth 4 gold answer: No

Context: In the bakery, the head baker knows that if she uses high-quality flour, then if she kneads the dough thoroughly, the bread will rise perfectly and have an airy texture.

Question: The head baker kneaded the dough thoroughly, but the bread did not rise perfectly. Did the head baker use high-quality flour?

Llama-3.1-8B-Instruct correct (No), 251 tokens

To determine if the head baker used high-quality flour, let's break down the given information and the outcome.

1. The head baker kneaded the dough thoroughly: This indicates that the head baker followed one of the necessary steps for making bread, which is kneading. However, kneading alone does not guarantee a perfect rise.

2. The bread did not rise perfectly: This outcome could be due to several factors, including the type of flour used, the yeast quality, temperature, and time.

Given that the head baker kneaded the dough thoroughly but the bread did not rise perfectly, we can infer that the issue might not be with the kneading process. However, we cannot directly conclude that the head baker used high-quality flour based on this information alone.

To determine if the head baker used high-quality flour, we would need more information about the outcome of the bread, such as its texture, taste, or other factors that could indicate the quality of the flour.

However, based on the given information, we can only say that the head baker followed one of the necessary steps for making bread, but the outcome was not as expected. Therefore, we cannot definitively conclude that the head baker used high-quality flour.

\boxed{No}

Fine-tuned on LogicDAG correct (No), 189 tokens

To determine if the head baker used high-quality flour, let's break down the given information and the rule:

1. The rule states that if the head baker uses high-quality flour, then if she kneads the dough thoroughly, the bread will rise perfectly and have an airy texture.
2. The head baker kneaded the dough thoroughly.
3. The bread did not rise perfectly.

Since the condition of kneading the dough thoroughly is met, we can apply the rule. The rule states that if high-quality flour is used, then the bread will rise perfectly. However, the bread did not rise perfectly, which contradicts the expected outcome.

Given that the outcome (perfectly risen bread) did not occur, and the condition (thorough kneading) is met, we can conclude that the antecedent (high-quality flour) must be false. Therefore, the head baker did not use high-quality flour.

\boxed{No}

Figure 19: A Multi-LogiEval item in propositional logic on Llama-3.1-8B-Instruct. Both models answer correctly. The fine-tuned model applies the nested conditional with the given kneading and modus tollens in 189 tokens, while the base model does not use the rule, cites outside factors such as yeast and temperature, and uses 251 tokens.

Multi-LogiEval, propositional logic, depth 3 gold answer: Yes

Context: Mark knew that if he finishes his errands, he'll have free time to relax. He remembered that his friend Stella got a new job and was supposed to start soon. Mark knew that if it's Stella's first day today, she'll be nervous but excited. Either Mark finished his errands, or Stella wasn't nervous and excited, or Mark finished errands and Stella wasn't nervous and excited.

Question: If it was Stella's first day at her new job, then did Mark have free time to relax?

Llama-3.1-8B-Instruct incorrect (No), 159 tokens

To determine if Mark had free time to relax, we need to analyze the given information step by step.

1. If it was Stella's first day at her new job, then she would be nervous and excited.
2. Mark knew that if he finishes his errands, he'll have free time to relax.
3. The statement "Either Mark finished his errands, or Stella wasn't nervous and excited" implies that if Stella was nervous and excited, then Mark did not finish his errands.

From step 1, we know that Stella was nervous and excited, which means from step 3, Mark did not finish his errands.

Since Mark did not finish his errands, he would not have free time to relax.

Therefore, the answer is: \boxed{No}

Fine-tuned on LogicDAG correct (Yes), 145 tokens

To solve this problem, let's break it down step by step:

1. If it was Stella's first day at her new job, then Stella is nervous and excited.
2. Given that Stella is nervous and excited, we can conclude that the statement "Stella wasn't nervous and excited" is false.
3. Since the statement "Either Mark finished his errands, or Stella wasn't nervous and excited" is true, and the second part of the disjunction is false, the first part of the disjunction must be true.
4. Therefore, Mark finished his errands.
5. Since Mark finished his errands, he has free time to relax.

The final answer is: \boxed{Yes}

Figure 20: A Multi-LogiEval item in propositional logic on Llama-3.1-8B-Instruct. The fine-tuned model applies disjunctive syllogism and modus ponens and answers correctly, while the base model misreads the disjunction as 'if Stella was nervous and excited, then Mark did not finish his errands' and then denies the antecedent.

System message (all five benchmarks)

You are a helpful AI assistant.

FOLIO

Premises:
1. ⟨*premise 1*⟩
2. ⟨*premise 2*⟩
…
Conclusion: ⟨*conclusion*⟩
Based ONLY on the premises, is the conclusion True, False, or Uncertain (cannot be determined)?
Reason step by step, then put your final answer in `\boxed{}` as one of: True, False, Uncertain.

Multi-LogiEval

Context: ⟨*context*⟩
Question: ⟨*question*⟩
Reason step by step, then put your final answer in `\boxed{}` as either Yes or No.

LogicBench (MCQA)

Context: ⟨*context*⟩
Question: ⟨*question*⟩
Choices:
A. ⟨*choice 1*⟩
…
D. ⟨*choice 4*⟩
Reason step by step, then put your final answer in `\boxed{}` as one of: A, B, C, D.

LogiQA 2.0

Passage: ⟨*passage*⟩
Question: ⟨*question*⟩
Choices:
A. ⟨*option 1*⟩
…
D. ⟨*option 4*⟩
Reason step by step, then put your final answer letter in `\boxed{}` (one of: A/B/C/D).

AGIEval LSAT-LR

⟨*passage*⟩Q: ⟨*question*⟩ Answer Choices: (A)⟨*option A*⟩ (B)⟨*option B*⟩ … (E)⟨*option E*⟩
Let's think step by step.
After reasoning, output only your final answer letter inside `\boxed{{}}` (one letter matching the labeled choices).

Figure 21: User messages of the evaluation harness for the five benchmarks with their own templates. ⟨…⟩ marks a field filled from the benchmark item.